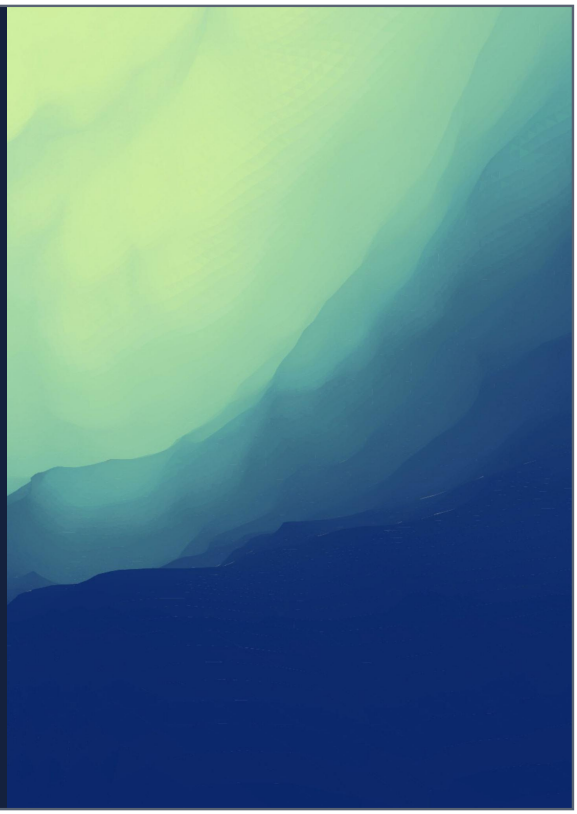
MODULE 06

Introduction to Software Architectures

Complexity, quality attributes, architectural styles, web architectures and microservices

```
>>> layers = ["routes", "services", "repositories"]
```



AIN3005 · Advanced Python Programming

Binnur Kurt, PhD · Bahçeşehir University · AI Engineering

Learning Objectives

By the end of this module, you will be able to:

1. Explain how partitioning, knowledge and abstraction help developers tame software complexity
2. Define software architecture and relate architectural choices to quality attributes and trade-offs
3. Document an architecture with views, the C4 model and architecture decision records (ADRs)
4. Distinguish architecture from design and apply core architectural principles and styles
5. Compare web architectures — server-side MVC, SPAs, RESTful data services — and choose a data store
6. Describe the evolutionary architect's role: principles, practices and service boundaries
7. Explain microservices — characteristics, benefits, challenges, decomposition — and the fallacies of distributed computing
8. Contrast service-oriented architecture (services, ESB, registry) with microservices

MODULE 06 · SOFTWARE ARCHITECTURES

Module Roadmap

01 **Software Complexity and Architecture**
Why systems need structure — and the weapons developers use

02 **A Tale of Two Systems**
Same features, different architectures, different qualities

03 **Defining Software Architecture**
Definitions, partitioning and why every program has one

04 **Quality Attributes and Documentation**
The qualities architecture must deliver — and how to write it down

05 **Architecture vs Design and Architectural Principles**
Where architecture ends, which principles guide it, and the styles it uses

06 **Web Architectures**
From request–response to single-page apps, RESTful data services and data stores

07 **The Evolutionary Architect**
Guiding a system that never stops changing

08 **Microservices**
Decomposing one application into small, independently deployable services

09 **Service-Oriented Architecture**
Integrating enterprise applications through shared, standards-based services

10 **Microservices vs SOA**
Same service idea, different scope, granularity, sharing and integration

01

Software Complexity and Architecture

Why systems need structure — and the weapons developers use

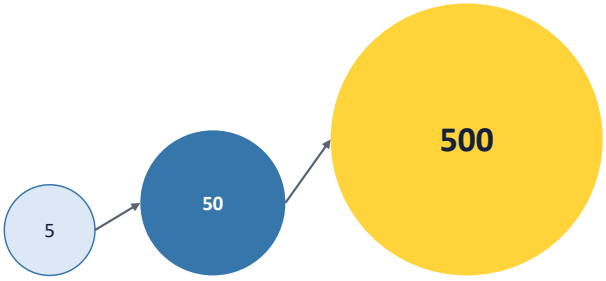
- Complexity and scale keep growing
- Tangible and intangible weapons
- Partitioning, knowledge and abstraction

Software Keeps Growing

- Decade after decade, software grows by **orders of magnitude** in size and complexity
- Today's systems are arguably the largest, most complex things people build
- Developers battle two ever-stronger foes: **complexity** and **scale**

Thought experiment

Imagine basketball scaling the same way: 5 players on the court one decade, then 50, then 500 — with the same rules and the same referees.

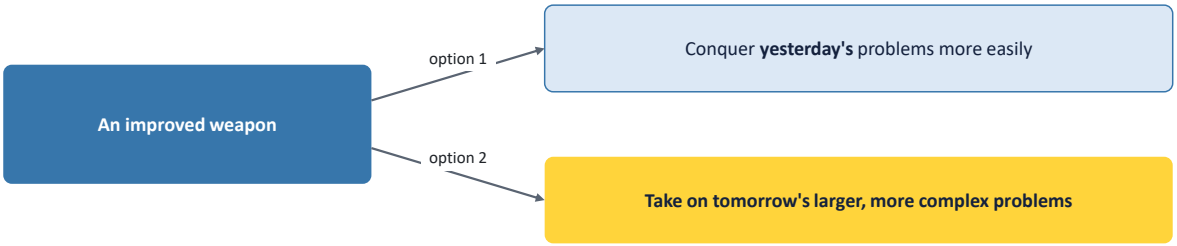


players on the court, decade after decade

The Developer's Weapons

Tangible weapons
- IDEs, debuggers and profilers
- Programming languages and libraries
- Version control, CI pipelines, test runners

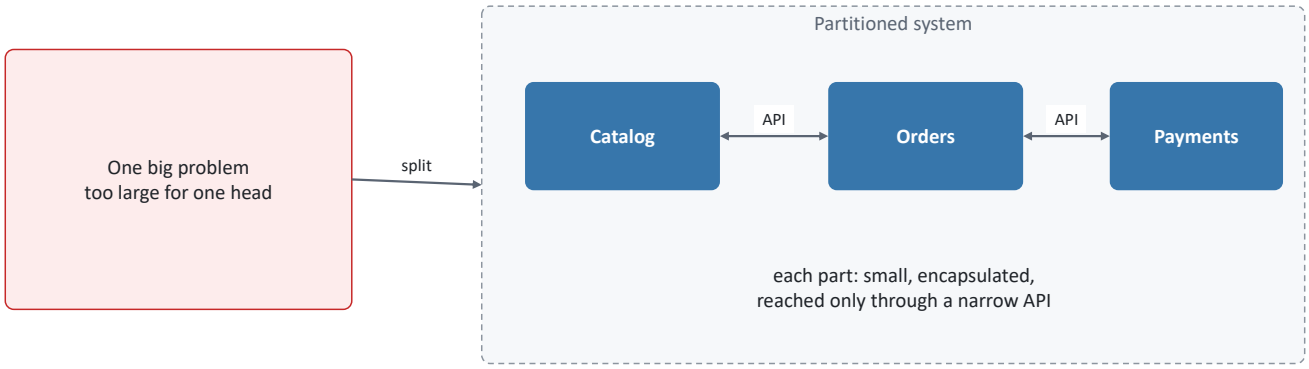
Intangible weapons — bigger impact
- **Partitioning**: divide and conquer
- **Knowledge**: reuse what worked before
- **Abstraction**: hide details to shrink problems



Key idea

We are no smarter than earlier generations — our weapons are better.

Partitioning: Divide and Conquer



- Works only if the parts are **small enough** for one person to solve…
- …and we can still **reason about how the parts assemble** into a whole
- **Encapsulated** parts let us forget their insides and track fewer details when composing

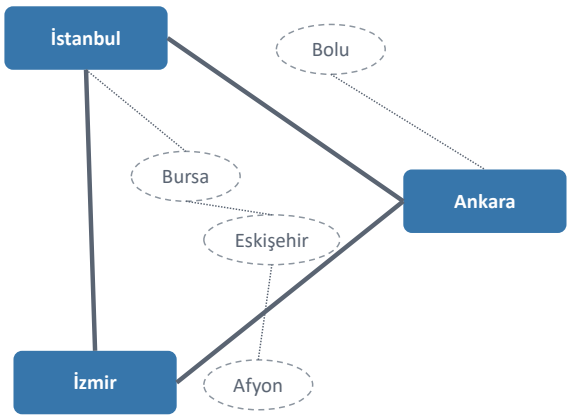
Knowledge: Reusing What Worked

	Implicit (know-how)	Explicit (written down)
Specific	"These two libraries work well together"	Project README, design documents, decision records
General	A feel for a good table layout or API shape	Books, pattern catalogues, lectures, source code

- Developers solve current problems with knowledge of **prior problems**
- It comes as books, lectures, patterns, code, design documents, whiteboard sketches

> ✓ **Tip**
> Architectural styles and patterns (section 5) are packaged, explicit knowledge: someone already paid for the lesson.

Abstraction: Shrinking the Problem



- Abstraction hides details, so the problem **shrinks**
- Driving İstanbul → Ankara? Consider only **highways**: far fewer options
- In code: a function signature, a `Protocol`, a box on a diagram

Key idea

`open(path)` lets you ignore disks, file systems and drivers — you reason about a stream of text.

DISCUSS

Spot the Weapon

Which weapon — **partitioning**, **knowledge** or **abstraction** — does each scenario use *primarily*?

#	Scenario
1	A team splits a 5,000-line `app.py` into `catalog`, `orders` and `payments` packages
2	A developer applies the Repository pattern she read about to isolate database code
3	An architecture diagram shows the whole payment provider as a single box
4	A new hire reads the project's decision log before replacing the database
5	`httpx.get(url)` lets you ignore sockets, TLS and HTTP parsing

ANSWER

Spot the Weapon

#	Weapon	Why
1	**Partitioning**	Divides the code into parts with small public interfaces
2	**Knowledge**	Reuses an explicit, general solution (a published pattern)
3	**Abstraction**	Hides the provider's internals to keep the picture small
4	**Knowledge**	Explicit, specific knowledge: why past decisions were made
5	**Abstraction**	The library hides the networking details behind one call

> **Note**
> Real designs use all three at once: the Repository pattern (knowledge) is an abstraction that partitions data access from business rules.

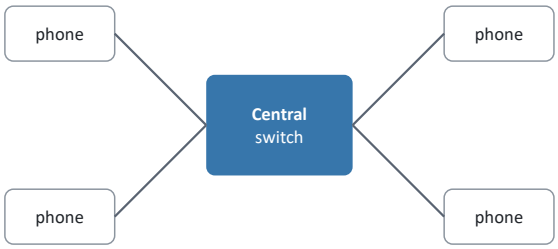
02

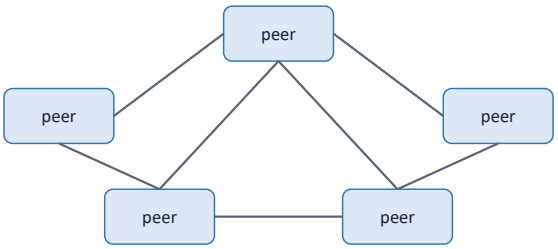
A Tale of Two Systems

Same features, different architectures, different qualities

- Telephone network vs Skype
- Design exercise: searching mail logs
- Rackspace: one system, three architectures

Same Feature, Different Architecture





	Plain old telephone system	Skype (original design)
Feature	Call a subscriber	Call a subscriber
Architecture	Centralized hardware switches	Peer-to-peer software on users' PCs
Good qualities	Works during power outages · very reliable · emergency calls carry location	Scales without new central hardware · easy to add features (e.g. video calls)

Key idea

Users ask for **features**; architects pay most attention to the **qualities** that arise from architecture choices.

DISCUSS

Design a Mail-Log Search System

The situation

- You are a hosting provider renting out mail servers
- Customers report problems ("my email bounced")
- Support staff diagnose them from the mail **log files**

Why it is hard

- **Hundreds** of servers
- **Gigabytes** of logs every day
- Collecting and searching logs takes time

Your task

- Sketch an architecture that lets support staff search the logs
- Which qualities does your design favour — and sacrifice?

Hints

- Central collection of logs?
- Distributed searching of logs?
- Pre-process logs to speed up queries?

Surprise: The System Is Real — Rackspace

Real experience

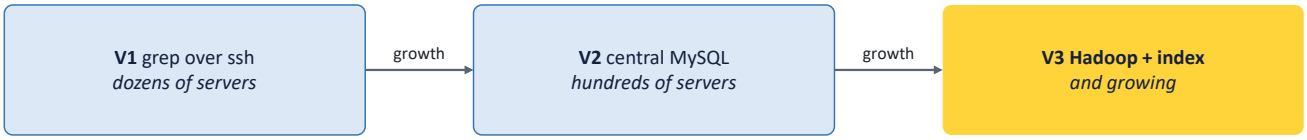
- Rackspace: a large hosting provider
- Huge growth in customers, mail servers — and problems
- Redesigned 3 major times (6 versions in total)

Same functionality

- All three versions answer the same questions
- …but with **different architectures**

Why it is instructive

- Building one system three times is rare and expensive
- Only the architecture changed
- So we see its effect on **quality attributes**



Sources: T. Hoff, highscalability.com (2008); G. Fairbanks, Just Enough Software Architecture (2010)

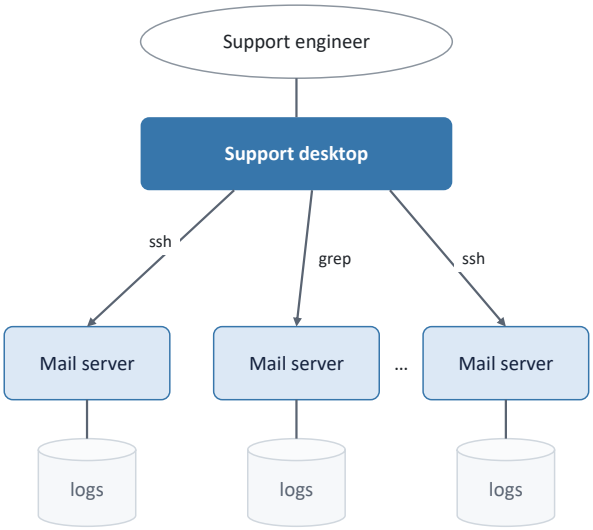
Rackspace V1: grep over ssh

Architecture

- Support engineer's desktop computer
- `ssh` connections to every mail server
- Log files stay **local** on each server

Procedure

- Write the query as a `grep` expression
- A script runs it via `ssh` on every server
- Results are aggregated on the desktop



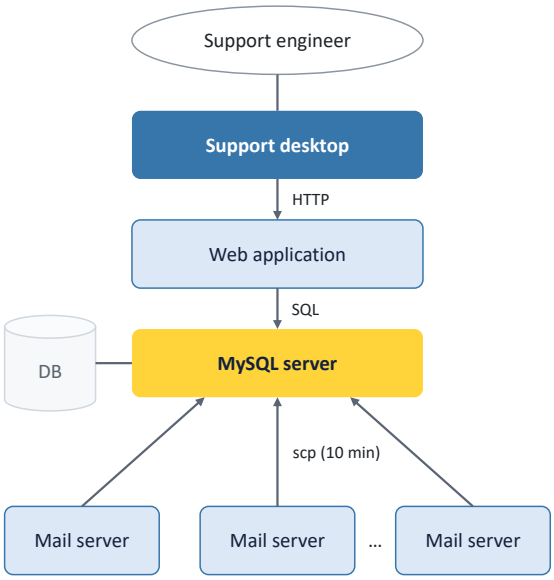
Rackspace V2: A Central MySQL Database

Architecture

- Support desktop and a **web application**
- Central **MySQL** database
- `scp` transfers logs from the mail servers

Procedure

- Every 10 minutes, send new logs to the MySQL server
- Parse and load them; combine with older logs
- Queries are answered from the database



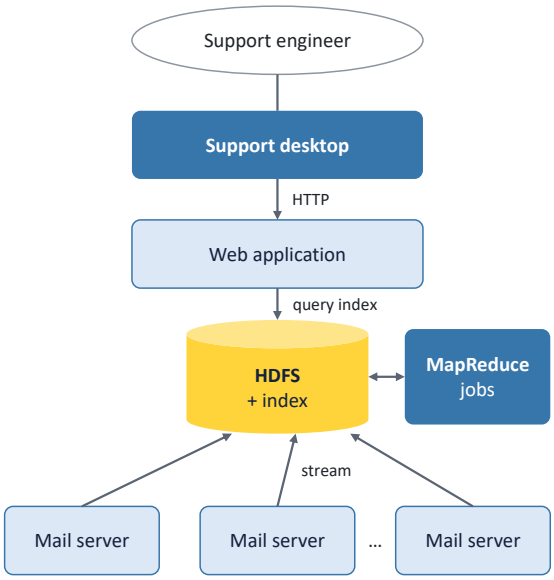
Rackspace V3: Hadoop and MapReduce

Architecture

- Support desktop and web application
- **Distributed file system** (HDFS)
- **MapReduce** job cluster builds an index

Procedure

- Log data streams continuously into HDFS
- Every 10 minutes a MapReduce job indexes new logs
- The web app queries the index



Rackspace: Quality Attribute Trade-Offs

Quality	V1: grep + ssh	V2: MySQL	V3: Hadoop
Data freshness	Current data	Up to 10 minutes old	10–20 minutes old
Scalability	Mail servers slow down (dozens of servers)	MySQL speed and stability problems (hundreds)	No problems yet
Ad hoc query ease	Regular expression	SQL expression	MapReduce program
Operations effort	Minimal: a script	One database server	A whole cluster

Key idea

No version is simply *better*: each one wins on some qualities and loses on others. Architecture is choosing which qualities matter most **now**.

QUIZ

Quick Check: Rackspace Trade-Offs

A customer phones: "An email to me bounced two minutes ago — why?" Which Rackspace version can find the answer in the logs right away?

A. V1 only (grep over ssh)

B. V2 only (central MySQL)

C. V3 only (Hadoop index)

D. V2 and V3

ANSWER

Quick Check: Rackspace Trade-Offs

A customer phones: "An email to me bounced two minutes ago — why?" Which Rackspace version can find the answer in the logs right away?

- **A** V1 only (grep over ssh)
- **B** V2 only (central MySQL)
- **C** V3 only (Hadoop index)
- **D** V2 and V3

Why

V1 searches the live log files on every server. V2 loads logs every 10 minutes and V3 indexes them every 10 minutes, so a two-minute-old bounce may not be there yet.

2 · A TALE OF TWO SYSTEMS

Lessons from Rackspace

1. Same functions, different qualities

Functionality alone does not determine architecture — qualities do

2. Growth drives redesign

V1 and V2 were suitable until the scale changed around them

3. Every choice costs something

Freshness, scalability, query ease and operations effort pull against each other

4. Architecture is expensive to change

Each new version was a major project, not a refactoring

Key idea

Choose the architecture for the qualities you need at your scale — and expect to evolve it (the evolutionary architect, section 7).

03

Defining Software Architecture

Definitions, partitioning and why every program has one

- Architect, architecting, architecture
- Standard definitions
- Suitable vs unsuitable architectures

Architect, Architecting, Architecture



Architect

A **job title** or role — sometimes one person, often shared by the whole team

Architecting

A **process**: making and revising architectural decisions throughout development

Architecture

An **engineering artifact**: the structures of the system — documented or not

✗ **Common misconception**

Keep them apart: a system with no one titled *architect* still has an architecture, and architecting happens whenever anyone makes a structural decision.

Two Definitions of Software Architecture

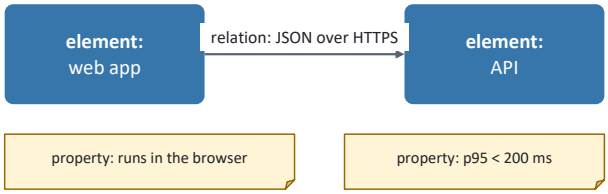
SEI — Clements et al., Documenting Software Architectures (2010)

The software architecture of a computing system is the set of **structures** needed to reason about the system, which comprise software **elements**, **relations** among them, and **properties** of both.

IEEE 1471-2000 (today: ISO/IEC/IEEE 42010)

The fundamental **organization** of a system embodied in its **components**, their **relationships** to each other and to the environment, and the **principles** guiding its design and evolution.

- Loosely: the **big picture** — the macroscopic organization of the system
- Open question: why are some detailed decisions architectural and others not? (section 5)



Architecture as Prudent Partitioning



Emerging from the interactions: functionality and quality attributes (security, modifiability, performance …)

- Architecture: the **prudent partitioning** of a whole into parts with specific relations
- Parts interact through carefully crafted interfaces: **minimal, stable** information
- The larger the system and the more demanding its qualities, the more critical this is

Every Program Has an Architecture

- Every program has an architecture — but not every architecture **suits** the program
- Requirements are **functional** (what it does) and **quality** needs (how well)
- Different architectures favour different needs, e.g. high throughput vs interactivity
- Judge architectures as **suitable** or **unsuitable**, not *good* or *bad*

Just enough architecture

Architecture is hard to change later. That is no excuse for Big Design Up Front (**BDUF**) — but it is a reason to think *enough* before coding.

Tip

Invest design effort in proportion to **risk**: the riskier the qualities, the more architecture work pays off (Fairbanks, 2010).

What If You Don't Think Architecturally?

Local optimisation

Developers miss the big picture: poor choices of frameworks, languages, data stores

Hero dependence

Success hinges on virtuosos — how many Guido van Rossums or Jeff Deans does your team have?

Poor communication

Idiosyncratic notations and fuzzy semantics on every whiteboard

Shallow analysis

Ad hoc designs from first principles; little attention to trade-offs and rationale

Patterns ignored

Known-good designs are missed — or chosen incorrectly

The remedy

Just enough architecture: make risks, qualities and decisions explicit

QUIZ

Quick Check: Does It Have an Architecture?

A student team ships a working prototype without any diagrams, documents or anyone called "architect". Which statement is true?

- **A** It has no architecture until someone documents one
- **B** It has an architecture — perhaps an accidental and unsuitable one
- **C** Only systems running on many servers have an architecture
- **D** Its architecture is simply its choice of web framework

ANSWER

Quick Check: Does It Have an Architecture?

A student team ships a working prototype without any diagrams, documents or anyone called "architect". Which statement is true?

- **A** It has no architecture until someone documents one
- **B** It has an architecture — perhaps an accidental and unsuitable one
- **C** Only systems running on many servers have an architecture
- **D** Its architecture is simply its choice of web framework

★ **Why**

Every program has structures — elements, relations and properties — whether or not anyone designed or wrote them down. Documentation describes an architecture; it does not create it.

04

Quality Attributes and Documentation

The qualities architecture must deliver — and how to write it down

- Quality attributes, scenarios and trade-offs
- What architecture documentation is for
- Views, the C4 model and decision records (ADRs)

Correct Is Not Enough

For nearly all systems, quality attributes matter as much as computing the right answer. A correct result is useless if…

…it takes too long

Performance: users give up, batch jobs miss their deadlines

…the system is down

Availability: the answer never reaches anyone

…the wrong people see it

Security: results leak to competitors — or attackers

★ **Key idea**

Architecture is where these concerns are addressed: you cannot bolt performance, availability or security onto a finished system.

What Each Quality Attribute Demands

If you need…	…the architecture must
Performance	Exploit parallelism by splitting work into cooperating processes; limit inter-process and network traffic; estimate latency and throughput; find bottlenecks
Accuracy	Define data elements carefully and control how their values flow through the system
Security	Restrict which parts may use and talk to which; guard the parts where intrusion hurts most; add trusted elements (e.g. an auth service)
Modifiability, portability	Separate concerns so a change to one element does not ripple across the system
Incremental deployment	Keep dependencies untangled — avoid *nothing works until everything works*

Making Quality Attributes Measurable

"The system must be fast" cannot be tested. A **quality attribute scenario** can:



Quality	Example scenario
Availability	A database node crashes during business hours → service continues; < 1 min of failed requests
Modifiability	A developer adds a payment method → done in < 2 days, touching one service

Qualities Pull Against Each Other

Improving…	…often costs	Example
Performance (caches, indexes)	Freshness, modifiability	Rackspace V3 answers from a 10–20 minute old index
Security (encryption, checks)	Performance, usability	TLS plus token validation on every request
Availability (replicas)	Consistency, cost	CAP: during a network partition, choose consistency or availability
Modifiability (layers, interfaces)	Performance, simplicity	Extra hops: routes → services → repositories
Scalability (distribution)	Simplicity, testability	Microservices (section 8)

★ **Key idea**

Architecture is choosing which qualities to favour — and making the trade-off **explicit**.

DISCUSS

Trade-Off Discussion: Three Systems

For each system, rank its **three most important** quality attributes and name **one trade-off** you would accept.

Course registration

- Opens for all students at 10:00 on enrolment day
- Seats are limited per course

Hospital patient records

- Doctors read and update records around the clock
- Highly sensitive personal data

Multiplayer online game

- Thousands of players in real-time matches
- A glitch costs players, not lives

✓ **Tip**

Think in scenarios: who does what, under which conditions, and how would you measure success?

Why Document an Architecture?

Education

Introduces new team members, external analysts, a new architect — or a customer seeing the solution for the first time

Communication

The primary vehicle among stakeholders: developers, testers, operators, managers, customers

Analysis and construction

Tells implementers what to build; lets evaluators check performance, security, availability …

Key idea

Documentation helps architects **make** decisions, tells developers how to **carry them out**, and **records** them for the system's future caretakers.

Documenting How Qualities Are Achieved



Drivers

Which quality requirements shaped the design?

Solutions

Which approaches were chosen to satisfy them?

Rationale

Why do they deliver the qualities — and what did they trade away?

Approach	Typically promotes
Client–server	Scalability
Layering	Portability, modifiability
Information hiding	Modifiability
Services	Interoperability

Tip

Traceability: look up a requirement such as *AVAIL-3* by name and find every place in the architecture that satisfies it.

Quality Bounds and a Shared Vocabulary

Elements that provide a service carry **quality bounds** in their interface documentation — a quality-of-service contract or service-level objective:

catalog-api.slo.yaml

```yaml
service: catalog-api
latency_p95_ms: 200
availability: "99.9%"
max_requests_per_second: 500
auth: OAuth 2.0 bearer token
```

Security

security levels, authenticated users, audit trails, firewalls

Performance

buffer capacities, deadlines, event rates, clocks and timers

Availability

MTBF, failover, redundant elements, critical vs non-critical processes

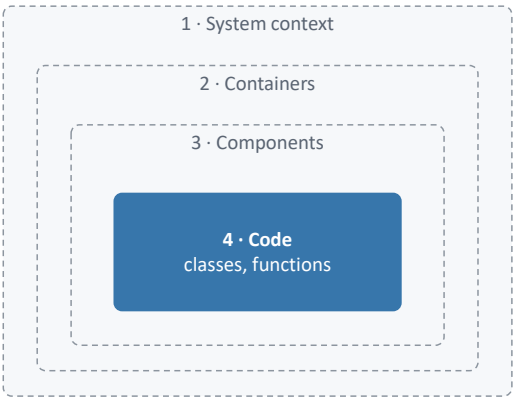
Views: One System, Several Structures

View type	Elements and relations	Answers	Python example
Module	Modules, packages, classes; *uses*, *is-a*	How is the code organised? What may import what?	`routes/`, `services/`, `repositories/` packages
Component-and-connector	Processes, services, stores; HTTP calls, messages	How does it behave at run time?	Uvicorn workers calling MongoDB and a payment API
Allocation	Software mapped to hardware, files, teams	Where does it run? Who owns it?	Docker containers on cloud VMs; one team per service

Note

No single diagram shows everything: choose the views your stakeholders need (Clements et al., 2010; Kruchten's 4+1 views, 1995).

The C4 Model: Four Levels of Zoom



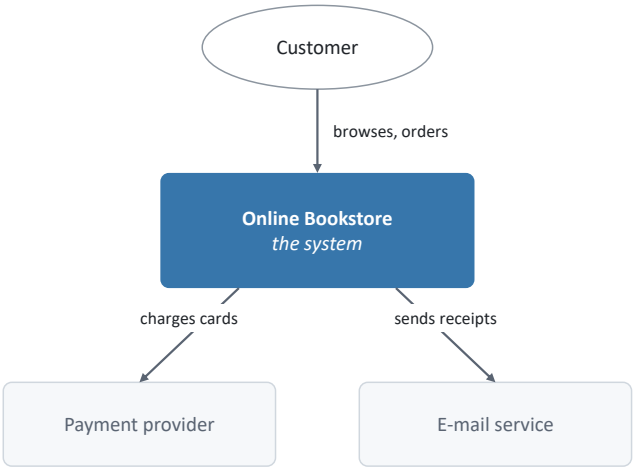
Level	Shows	For
Context	The system, its users and neighbouring systems	Everyone
Container	Runnable/deployable units: SPA, API, database	Developers, ops
Component	Building blocks inside one container	Its developers
Code	Classes and functions — often generated or skipped	Developers

Watch out

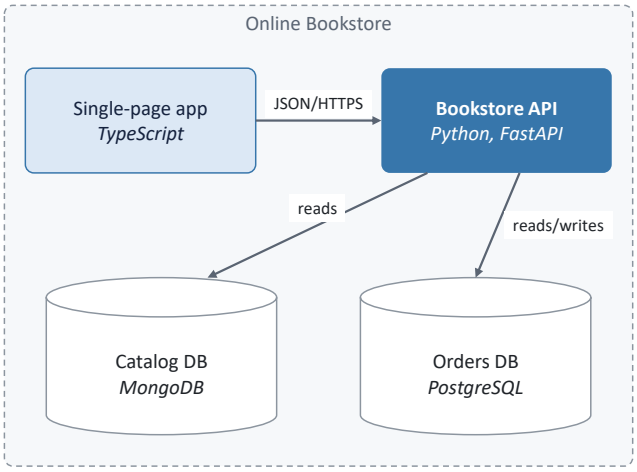
A C4 *container* is any separately runnable unit — not necessarily a Docker container.

C4 in Practice: Context and Containers

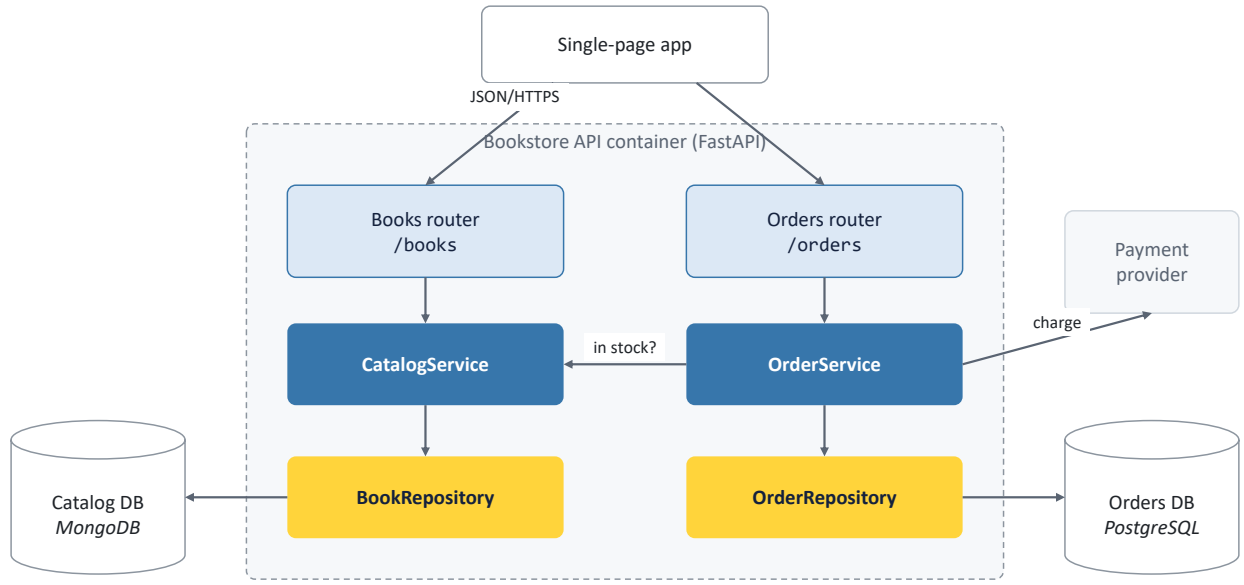
Level 1 — System context



Level 2 — Containers



C4 Level 3: Components of the API



Architecture Decision Records (ADRs)

- An **ADR** records one architecturally significant decision (Nygard, 2011)
- Short Markdown files, numbered, stored **in the repository** with the code
- Accepted ADRs are not edited — a new ADR **supersedes** an old one
- Answers the future question: *why on earth did they do that?*

```
docs/adr/
├── 0001-record-architecture-decisions.md
├── 0002-use-fastapi-for-the-api.md
└── 0003-use-mongodb-for-the-catalog.md
```

Title and status
`3. Use MongoDB for the catalog` — proposed, accepted, superseded

Context
The forces at play: requirements, constraints, qualities

Decision
What we will do, in full sentences

Consequences
What becomes easier — and what becomes harder

An Example ADR

docs/adr/0003-use-mongodb-for-the-catalog.md

```
# 3. Use MongoDB for the book catalog

Status: Accepted

## Context
Books have very different attributes (print, e-book, audio).
The catalog is read far more often than it is written, and the
SPA consumes JSON. The team knows PyMongo (Module 7).

## Decision
Store the catalog as documents in a MongoDB replica set,
accessed only through `BookRepository`.

## Consequences
+ Flexible schema; JSON end to end; reads scale with replicas.
- No joins across collections; reports need an export job.
- Orders stay in PostgreSQL for ACID transactions (ADR 4).
```


TRY IT

Write an ADR for Rackspace V3

You are on the Rackspace team in 2008. Write the ADR that records the move from the central MySQL design (V2) to Hadoop (V3).

- Use the sections **Status**, **Context**, **Decision**, **Consequences**
- Name at least **two negative** consequences
- Say which earlier ADR it **supersedes**
- Keep it under 15 lines

Tip

Hint: revisit the trade-off table of section 2 — freshness, scalability, query ease, operations.

ANSWER

Write an ADR for Rackspace V3

docs/adr/0007-process-mail-logs-with-hadoop.md

```
# 7. Process mail logs with Hadoop MapReduce

Status: Accepted (supersedes ADR 4: central MySQL log store)

## Context
Hundreds of mail servers produce gigabytes of logs per day. The
MySQL server that loads them every 10 minutes has speed and
stability problems and cannot grow beyond one machine.

## Decision
Stream logs into HDFS. Every 10 minutes a MapReduce job builds
a search index that the support web app queries.

## Consequences
+ Scales horizontally by adding commodity machines.
- Answers use data that is 10-20 minutes old.
- Ad hoc questions need a MapReduce program instead of SQL.
- The team must learn and operate a Hadoop cluster.
```


QUIZ

Quick Check: C4 Levels

Your diagram shows the single-page app, the FastAPI application and two databases as separate boxes, connected by labelled protocols. Which C4 level is it?

- A Level 1 — System context
- B Level 2 — Container
- C Level 3 — Component
- D Level 4 — Code

ANSWER

Quick Check: C4 Levels

Your diagram shows the single-page app, the FastAPI application and two databases as separate boxes, connected by labelled protocols. Which C4 level is it?

- A — Level 1 — System context
- **B — Level 2 — Container**
- C — Level 3 — Component
- D — Level 4 — Code

Why

Separately runnable or deployable units (apps, APIs, databases) and the protocols between them are exactly what a container diagram shows.

05

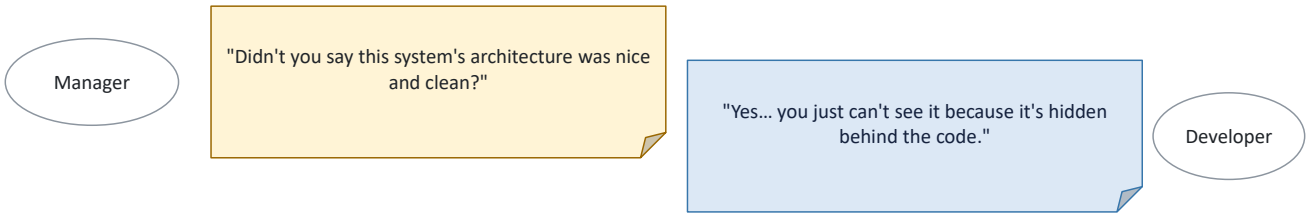
Architecture vs Design and Architectural Principles

Where architecture ends, which principles guide it, and the styles it uses

- Architectural vs non-architectural decisions
- Architect and designer
- Architectural principles
- Architectural styles at a glance

DISCUSS

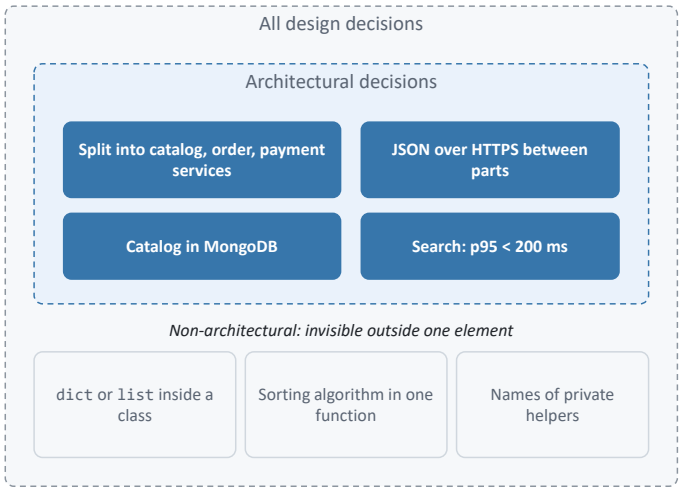
Warm-Up: Architecture or Design?



- What, if anything, makes architecture different from design?
- Can a good architecture make up for a bad design — or the other way round?
- Does object-oriented design (Module 1) have anything to do with architecture?
- Which documents does an architect produce, and for whom?

5 · ARCHITECTURE VS DESIGN AND ARCHITECTURAL PRINCIPLES

Not All Design Is Architecture



- **Architectural** decisions let the system meet its **quality** and behavioural requirements
- Decisions **invisible outside an element** are non-architectural: its data structures, its algorithms
- Many decisions are deliberately left to designers and implementers

Can Architectural Decisions Be Detailed?

A detailed — yet architectural — statement

"The element…

- delivers its result through **this published interface**
- is **thread-safe**
- puts **no more than three messages** on the network per invocation
- returns its answer in **less than 20 ms**"*

- Also detailed, still architectural: specific **protocols**, **data contracts** (JSON Schema, OpenAPI), technology **standards**
- Made for interoperability, scalability, extensibility …

★ **Key idea**

The test is not the level of detail but **visibility**: does it matter outside the element?

QUICK CHECK

Architecture or Design? Classify Decisions

Mark each decision **A** (architectural) or **D** (non-architectural design) — and say why.

#	Decision
1	All parts exchange JSON over HTTPS
2	`OrderService` caches prices in a `dict` rather than a `list`
3	The catalog answers 95% of queries within 200 ms
4	Search results are sorted with `sorted(..., key=...)`
5	The system is split into catalog, order and payment services
6	The internal class structure of the payment service
7	The payment component must be thread-safe
8	Catalog data in MongoDB, orders in PostgreSQL

ANSWER

Architecture or Design? Classify Decisions

#		Why
1	**A**	Interoperability: every communicating element depends on it
2	D	Invisible outside the class (unless it breaks a memory budget)
3	**A**	A quality bound published on an element's interface
4	D	An algorithm choice local to one function
5	**A**	The partitioning itself: defining the services
6	D	Left to the service's designers — as long as its interface and qualities hold
7	**A**	An externally visible property that clients rely on
8	**A**	Affects consistency, scalability, operations and several elements

5 · ARCHITECTURE VS DESIGN AND ARCHITECTURAL PRINCIPLES

Architect and Designer: Different Focus

	Architect	Designer
Abstraction level	High and broad; focus on few details	Low and specific; focus on many details
Deliverables	System and subsystem plans, architectural prototype, ADRs	Component designs, code specifications
Area of focus	Quality attributes (non-functional requirements), risk management	Functional requirements
Typical question	Which parts, and how do they talk?	Which classes and algorithms inside a part?

Note

Roles blur: in agile teams one person often does both. The distinction is about **decisions**, not job titles.

Shared Principles: From Classes to Components

Abstraction

A box labelled *business logic* that everyone understands without its details

Encapsulation

A boundary that hides the details the abstraction ignores

Cohesion

Everything inside the boundary contributes to the abstraction

Coupling

Helpful inside a boundary; a hindrance across boundaries

Tip

Same ideas as object design in Module 1, applied to components and services. And like code, an architecture needs **refactoring** as requirements change.

Architectural Principles

Separation of concerns

UI, business logic and data access in different parts — the root of MVC

Dependency inversion

Depend on abstractions, not concretions (SOLID, Module 1)

Separate volatile from stable

Keep the often-changing UI apart from stable domain entities

Use component frameworks

Let a container run your components: Uvicorn or Gunicorn serve Flask and FastAPI apps

Simple, clear interfaces

Small, cohesive interfaces are easy to understand and use correctly

Coarse-grained remote interfaces

One network call per operation — not one per field

Keep Remote Interfaces Coarse-Grained

login_api.py

```python
from typing import Protocol


class ChattyLoginService(Protocol):  # 3 calls
    def set_user_name(self, name: str) -> None: ...

    def set_password(self, password: str) -> None: ...

    def perform_login(self) -> bool: ...


class LoginService(Protocol):  # 1 call
    def login(self, name: str, password: str) -> bool: ...
```

Interface	Round trips	Latency at 40 ms
Chatty	3	120 ms
Coarse-grained	1	40 ms

Watch out

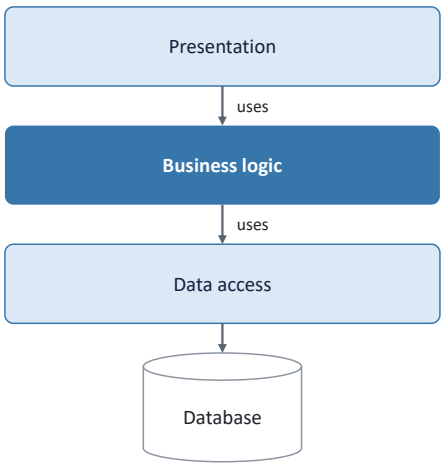
The chatty design also forces the server to remember **session state** between calls — which hurts scalability.

Architectural Styles at a Glance

Style	Core idea	Promotes	Watch out for
Layered	Each layer uses only the layer below	Modifiability, portability, testability	Extra hops; changes that cut through every layer
Client–server	Many clients request services from a shared server	Central data and control	Server bottleneck, single point of failure
MVC	Model, View, Controller separate UI concerns	Several views of one model	Overkill for tiny UIs
Pipes and filters	Independent filters connected by data streams	Reuse, composition, parallelism	Poor fit for interactive systems
Event-driven → Module 9	Producers publish events; consumers react asynchronously	Loose coupling, extensibility	Hard to trace; eventual consistency
Microservices → section 8	Small, independently deployable services owning their data	Independent deployment, team autonomy	Distributed-system complexity

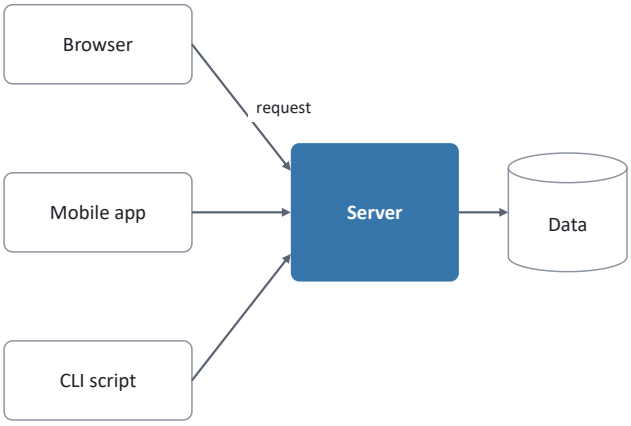
Layered and Client–Server Styles

Layered



A layer depends only on the layer below it

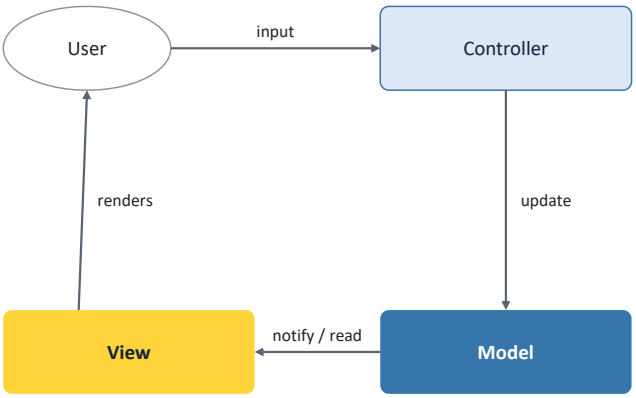
Client–server



Clients initiate; the shared server responds

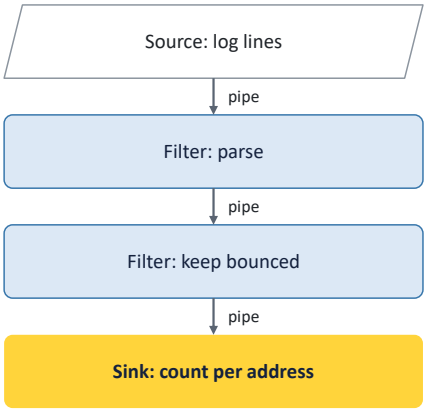
MVC and Pipes-and-Filters Styles

Model–View–Controller



Separate data, presentation and input handling

Pipes and filters



Independent steps connected by data streams

Pipes and Filters in Python

filters.py

```python
from collections import Counter
from collections.abc import Iterable, Iterator

Record = list[str]  # [time, address, status]


def parse(lines: Iterable[str]) -> Iterator[Record]:
    return (line.split() for line in lines)


def bounced(records: Iterable[Record]) -> Iterator[Record]:
    return (rec for rec in records if rec[2] == "bounced")


def per_address(records: Iterable[Record]) -> Counter[str]:
    return Counter(rec[1] for rec in records)
```

main.py

```python
from filters import bounced, parse, per_address

log = ["08:01 ada@uni.edu ok",
       "08:02 bob@uni.edu bounced",
       "08:05 eve@uni.edu ok",
       "08:07 bob@uni.edu bounced"]
print(per_address(bounced(parse(log))))
```

OUTPUT

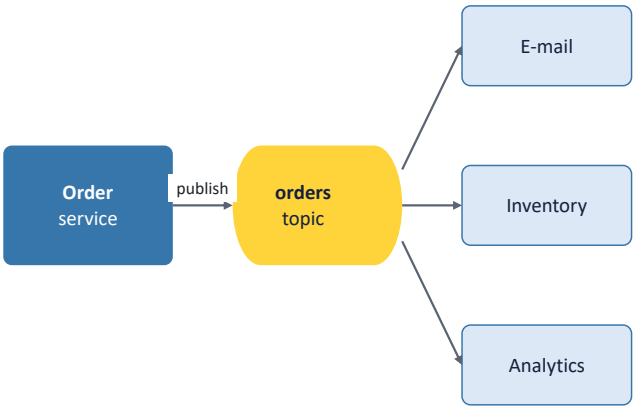
```
Counter({'bob@uni.edu': 2})
```

Key idea

Each generator is a **filter**; the iterator passed between them is the **pipe**. Nothing runs until the sink pulls data — lazy evaluation from Module 2.

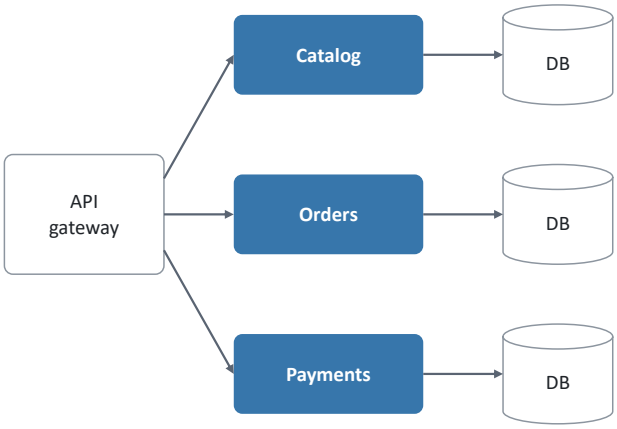
Event-Driven and Microservices Styles

Event-driven → Module 9



Producers do not know who reacts to their events

Microservices → section 8



Each service is deployed independently and owns its data

QUIZ

Quick Check: Pick the Style

A nightly job reads CSV exports, cleans them, validates each row, enriches it with exchange rates and loads the result into a data warehouse. Which style fits most naturally?

- **A** Layered
- **B** Pipes and filters
- **C** Model–View–Controller
- **D** Client–server

ANSWER

Quick Check: Pick the Style

A nightly job reads CSV exports, cleans them, validates each row, enriches it with exchange rates and loads the result into a data warehouse. Which style fits most naturally?

- **A** Layered
- **B** Pipes and filters ✓
- **C** Model–View–Controller
- **D** Client–server

Why

A one-way stream of records passing through independent transformation steps is the textbook case for pipes and filters; each step can be tested, reused and parallelised alone.

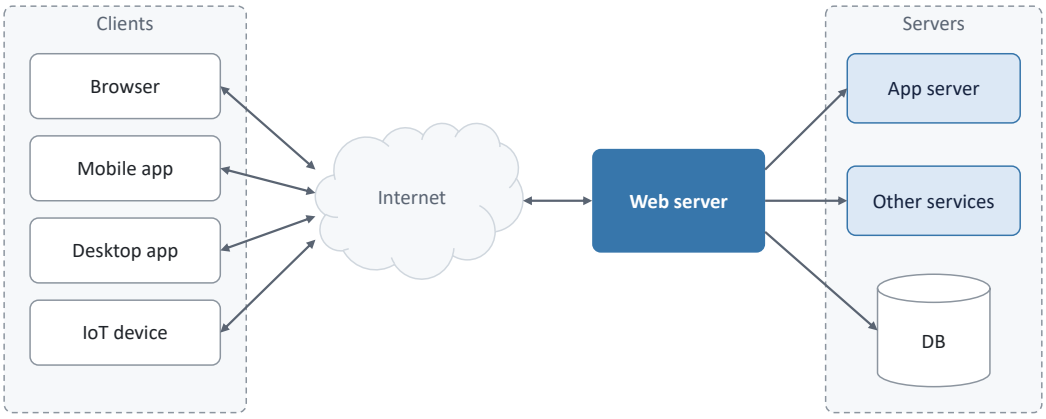
06

Web Architectures

From request–response to single-page apps, RESTful data services and data stores

- Request–response and the tiers behind it
- Server-side MVC, Ajax/fetch and single-page apps
- RESTful data services — and a layered Python service
- Choosing a data store: SQL, NoSQL, distributed ledgers

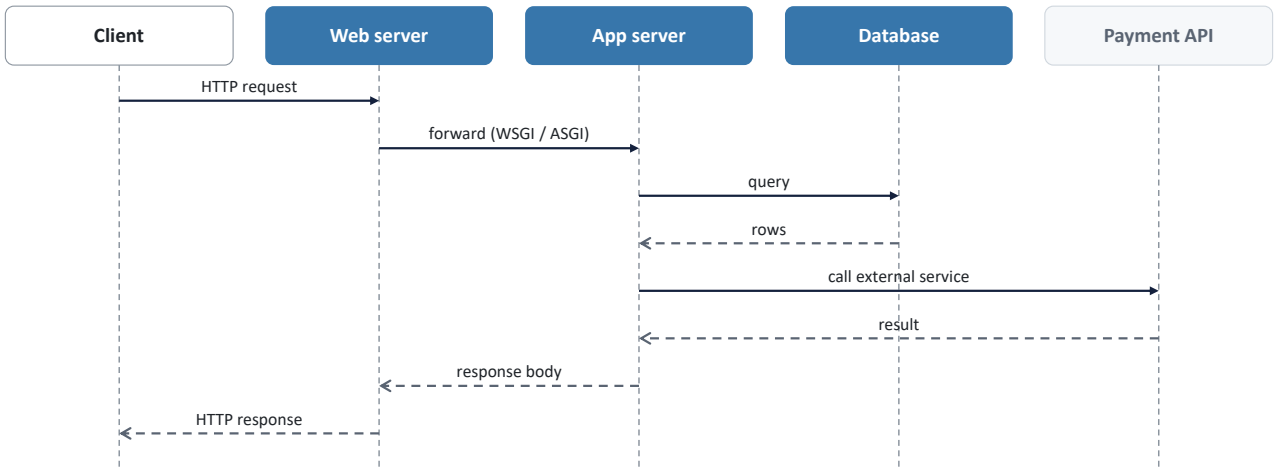
Web Architecture: Clients, Network, Servers



Key idea

Clients always **initiate**; servers may call other servers — databases, other services — to complete a request.

How a Web Server Handles a Request



① the client initiates · ② the server locates the resource and builds a response, possibly with help from other resources · ③ the response returns to the client

Anatomy of an HTTP Exchange

request

```
GET /books/42 HTTP/1.1
Host: api.bookstore.example
Accept: application/json
Authorization: Bearer eyJhbGciOi...
```

- **Method** + **path**: what to do with which resource
- **Headers**: metadata — format, credentials, caching

response

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=60

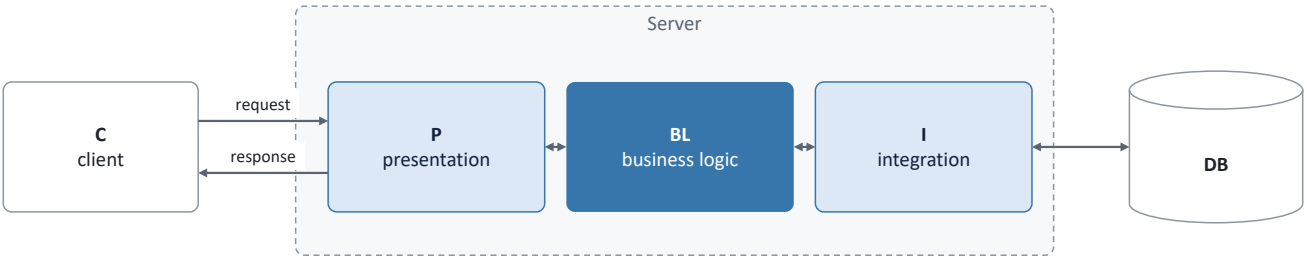
{"id": 42, "title": "SICP", "stock": 3}
```

- **Status code**: 2xx success · 4xx client error · 5xx server error
- **Body**: a *representation* of the resource

Key idea

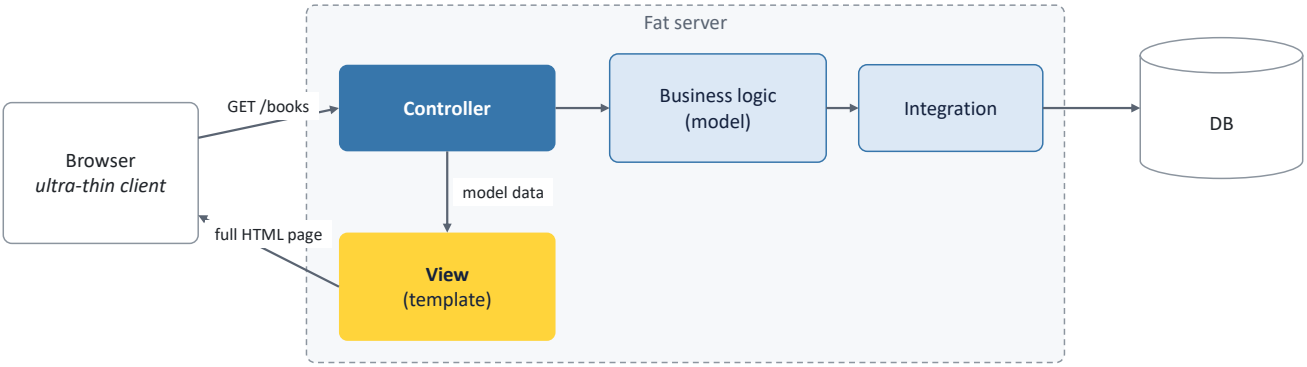
HTTP is **stateless**: every request carries everything the server needs to answer it.

The Tiers Behind a Request



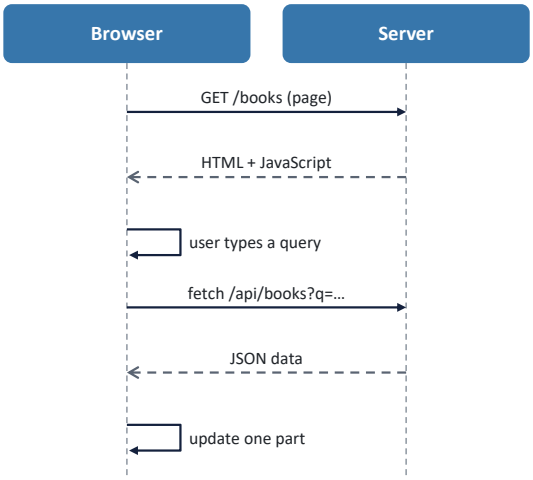
Tier	Responsibility	In a Python web app
P presentation	Turn HTTP into calls; render responses	Flask/FastAPI routes, Jinja2 templates
BL business logic	Rules and workflows of the domain	Service classes
I integration	Talk to databases and other systems	Repositories; PyMongo, SQLAlchemy, `httpx`

Server-Side MVC: Thin Client, Fat Server



- Every click requests a **full HTML page** rendered on the server (Jinja2 in Flask, Django templates)
- Simple, secure, search-engine friendly — but full page reloads and limited interactivity
- Still a sound choice for content sites and admin tools; *htmx* adds partial updates

Partial Page Updates: From Ajax to fetch



search.js

```
async function search(query) {
  const q = encodeURIComponent(query);
  const response = await fetch(`/api/books?q=${q}`);
  const books = await response.json();
  list.replaceChildren(...books.map(toItem));
}
```

- **Ajax** (2005): asynchronous `XMLHttpRequest` + partial page update
- Today: `fetch` with `async`/`await` — like `asyncio` (Module 5)
- The server sends **data (JSON)**, not HTML

Rich Clients: From Plug-ins to HTML5

Technology	Since	Fate
Java applets	1995	Browsers dropped plug-ins (2015–2017)
Adobe Flash	1996	End of life: December 2020
Microsoft Silverlight	2007	Support ended: October 2021
JavaFX	2008	Desktop only today (OpenJFX)
HTML5 + JavaScript	2014	**Runs everywhere, no plug-in**

Graphics
Canvas, SVG, WebGL, WebGPU

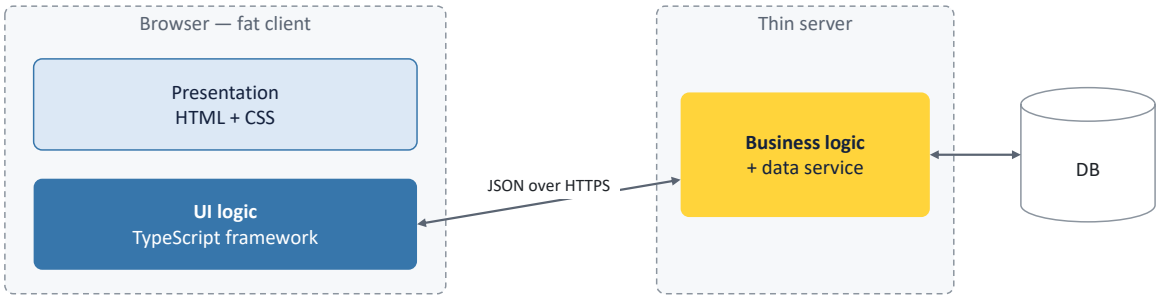
Connectivity
`fetch`, WebSockets, WebRTC

Offline and storage
IndexedDB, Service Workers

Performance
Web Workers, WebAssembly

Key idea
The browser became an application platform: the **presentation tier moved to the client**.

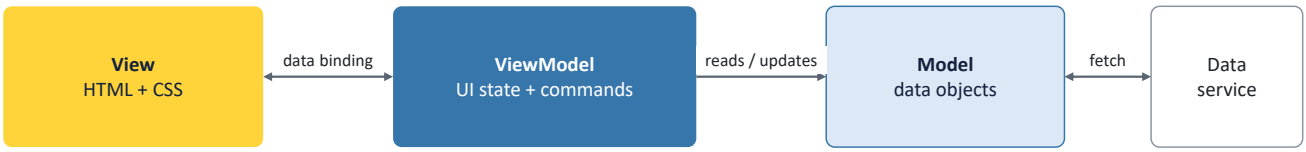
Single-Page Applications: Fat Client, Thin Server



- The browser downloads the app **once**, then exchanges only **data** (JSON)
- One data service (API) serves the web app, mobile apps and third parties
- Frameworks: React, Angular, Vue, Svelte — usually written in **TypeScript**

> **Watch out**
> Never trust the client: validate and authorise every request again on the server.

UI Logic in the Browser: MVVM



Model
Data and rules received from the data service

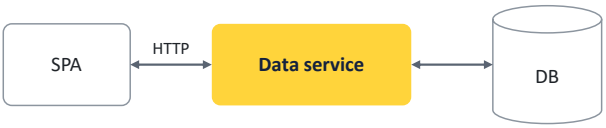
View
What the user sees: markup and styles, no logic

ViewModel
UI state and commands; data binding keeps the view in sync automatically

Language evolution: JavaScript ES5 → ES2015+ (classes, modules, `async`/`await`) → **TypeScript** (static types, like Python type hints)

Data Services: Resources over HTTP

Endpoint	Roughly in SQL	Meaning
`GET /books`	`SELECT … FROM books`	List books
`GET /books/42`	`SELECT … WHERE id = 42`	Read one book
`POST /books`	`INSERT`	Create; the server assigns the id
`PUT /books/42`	`UPDATE (or INSERT)`	Replace the whole resource
`PATCH /books/42`	`UPDATE some columns`	Partial update
`DELETE /books/42`	`DELETE`	Remove



Common misconception

REST is not *SQL over HTTP*: resources need not map one-to-one onto tables.

One Resource, Many Representations

request

```
GET /books/42 HTTP/1.1
Accept: application/json
```

response body

```
{"id": 42, "title": "SICP", "year": 1985}
```

request

```
GET /books/42 HTTP/1.1
Accept: text/csv
```

response body

```
id,title,year
42,SICP,1985
```

- A **resource** (book 42) can have several **representations**: JSON, XML, CSV, HTML, SVG …
- The client asks with `Accept`; the server answers with `Content-Type` — **content negotiation**
- JSON dominates web APIs today; XML lives on in enterprise and SOAP systems (section 9)

REST as an Architectural Style

REST (Fielding, 2000) is a set of **constraints**; each one induces qualities:

Constraint	Meaning	Promotes
Client–server	UI separated from data storage	Portability, independent evolution
Stateless	Each request carries all its context; no session on the server	Scalability, reliability
Cacheable	Responses label themselves cacheable or not	Performance, scalability
Uniform interface	Resources, URIs, standard methods, self-descriptive messages, hypermedia	Simplicity, decoupling
Layered system	Proxies, gateways, load balancers invisible to clients	Scalability, security
Code on demand (optional)	Server may send executable code (JavaScript)	Extensibility

> **Note**
> Designing and implementing REST APIs with Flask and FastAPI: Module 8.

QUIZ

Quick Check: REST Constraints

A web API keeps each user's shopping cart in the memory of the server process that handled the login, so every later request must reach that same process. Which REST constraint is violated?

- A Cacheable
- B Stateless
- C Layered system
- D Code on demand

ANSWER

Quick Check: REST Constraints

A web API keeps each user's shopping cart in the memory of the server process that handled the login, so every later request must reach that same process. Which REST constraint is violated?

A Cacheable	**B Stateless**
C Layered system	D Code on demand

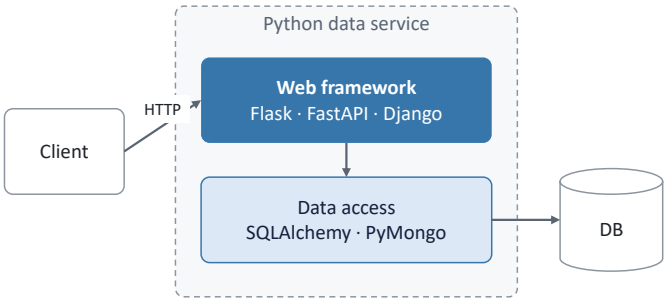
Why

Session state lives on the server, so requests are no longer self-contained. Load balancers need sticky sessions, and a crash of that process loses the cart.

6 · WEB ARCHITECTURES

RESTful Services on Any Platform

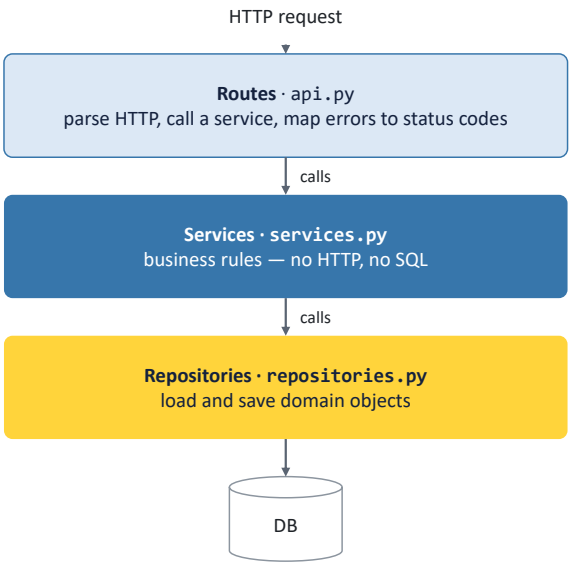
Platform	Typical frameworks
JavaScript (Node.js)	Express, NestJS
Java	Spring Boot, Jakarta EE (JAX-RS)
.NET	ASP.NET Core
Go	net/http, Gin
Python	**Flask, FastAPI, Django REST framework**



Key idea

The uniform interface hides the implementation: clients cannot tell — or care — which stack serves them.

Inside a Python Service: Three Layers




```
bookstore/
├── api.py            # routes (FastAPI)
├── services.py       # business rules
├── repositories.py   # data access
└── tests/
    ├── test_services.py
    └── test_api.py
```

Key idea

Dependencies point **downward** only: routes → services → repositories. Nothing below imports anything above.

Repository Layer: Hiding the Storage

repositories.py

```python
from dataclasses import dataclass
from typing import Protocol


@dataclass
class Book:
    book_id: int
    title: str
    stock: int


class BookRepository(Protocol):
    def get(self, book_id: int) -> Book | None: ...

    def save(self, book: Book) -> None: ...
```

repositories.py (continued)

```python
class InMemoryBookRepository:
    """Swap for a SQL or MongoDB version."""

    def __init__(self) -> None:
        self._rows: dict[int, Book] = {}

    def get(self, book_id: int) -> Book | None:
        return self._rows.get(book_id)

    def save(self, book: Book) -> None:
        self._rows[book.book_id] = book
```

Key idea

A `MongoBookRepository` or `SqlBookRepository` with the same two methods can replace it — the service never notices.

Service Layer: Business Rules Only

services.py

```python
from repositories import Book, BookRepository


class OutOfStock(Exception):
    """Business rule violated: not enough copies left."""


class OrderService:
    def __init__(self, books: BookRepository) -> None:
        self.books = books

    def order(self, book_id: int, quantity: int) -> Book:
        book = self.books.get(book_id)
        if book is None:
            raise LookupError(f"unknown book {book_id}")
        if quantity > book.stock:
            raise OutOfStock(book.title)
        book.stock -= quantity
        self.books.save(book)
        return book
```

Key idea

No `import fastapi`, no SQL: the service depends only on the `BookRepository` protocol — dependency inversion from Module 1.

- Domain problems are **exceptions**: `LookupError`, `OutOfStock`
- The route layer decides which status code they become

Route Layer: HTTP In, JSON Out

api.py

```python
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException

from repositories import InMemoryBookRepository
from services import OrderService, OutOfStock

app = FastAPI()
books = InMemoryBookRepository()
Service = Annotated[OrderService,
                    Depends(lambda: OrderService(books))]


@app.post("/books/{book_id}/orders")
def place_order(book_id: int, quantity: int,
                service: Service) -> dict[str, str | int]:
    try:
        book = service.order(book_id, quantity)
    except LookupError:
        raise HTTPException(404, "book not found") from None
    except OutOfStock:
        raise HTTPException(409, "not enough stock") from None
    return {"title": book.title, "stock_left": book.stock}
```

Key idea

Routes only **translate**: HTTP → method call → JSON or a status code.

Tip

`Depends` injects the service, so tests can swap it via `app.dependency_overrides`.

TERMINAL

```
$ uvicorn api:app --reload
```

Testing Each Layer in Isolation

tests/test_services.py

```python
from repositories import Book, InMemoryBookRepository
from services import OrderService


def test_order_reduces_stock() -> None:
    books = InMemoryBookRepository()
    books.save(Book(1, "SICP", stock=3))
    OrderService(books).order(1, quantity=2)
    assert books.get(1).stock == 1
```

Key idea

Service test: no web server, no database — plain objects, milliseconds to run.

tests/test_api.py

```python
from fastapi.testclient import TestClient

from api import app, books
from repositories import Book

client = TestClient(app)


def test_second_order_is_rejected() -> None:
    books.save(Book(1, "Clean Architecture", stock=3))
    first = client.post("/books/1/orders?quantity=2")
    second = client.post("/books/1/orders?quantity=2")
    assert first.json() == {"title": "Clean Architecture",
                            "stock_left": 1}
    assert second.status_code == 409
```

TERMINAL

```
$ python -m pytest -q
..                                  [100%]
2 passed in 0.34s
```


FIND THE BUG

Find the Layering Violations

app.py (Flask)

```python
@app.post("/books/<int:book_id>/orders")
def place_order(book_id: int):
    quantity = request.get_json()["quantity"]
    row = db.execute("SELECT title, stock FROM books WHERE id = ?",
                     (book_id,)).fetchone()
    if quantity > row["stock"]:
        return {"error": "not enough stock"}, 409
    db.execute("UPDATE books SET stock = stock - ? WHERE id = ?",
               (quantity, book_id))
    mailer.send("sales@bookstore.example", f"Sold {row['title']}")
    return {"title": row["title"], "stock_left": row["stock"] - quantity}
```

- Which lines belong in a **route**, a **service**, a **repository**?
- Why is this function hard to unit test?
- Find two bugs: a request that crashes with 500, and two requests for the last copy

ANSWER

Find the Layering Violations

Lines	Belongs in	Why
1–3, 11	Route	HTTP only: parse the request, shape the response
6–7	Service (`OrderService`)	The stock rule is business logic
4–5, 8–9	Repository	SQL is a storage detail
10	A notifier behind an interface	Infrastructure the service calls

Watch out

Unknown `book_id` → `row` is `None` → 500 instead of 404. Read-then-update is a **race condition** (Module 5): two requests can both see stock 1 and oversell. Fix with an atomic `UPDATE … WHERE id = ? AND stock >= ?` and check the affected row count.

6 · WEB ARCHITECTURES

Relational or NoSQL Data Service?

	Relational (SQL)	NoSQL
Data model	Tables and rows; a fixed schema	Documents, key–value pairs, wide columns, graphs; schema-flexible
Relationships	`JOIN`s across tables	Embedding and denormalisation; few or no joins
Consistency	ACID transactions	Often tunable or eventual; many now offer transactions
Scaling	Mostly vertical; not cluster-friendly	Horizontal; cluster-friendly by design
Examples	PostgreSQL, MySQL, SQLite	MongoDB, Redis, Cassandra, Neo4j, CouchDB

Key idea

Choose per data set — many systems use both (**polyglot persistence**). MongoDB in depth: Module 7.

Four Families of NoSQL Stores

Key–value

`cart:ada` → `{...}`

Fast look-ups by key: caches, sessions, shopping carts

Redis, DynamoDB, Memcached

Document

`{"title": "SICP", "authors": [...]}`

JSON-like documents with rich queries

MongoDB, CouchDB

Wide-column

Rows with dynamic columns, partitioned across nodes

Huge write volumes, time series

Cassandra, HBase

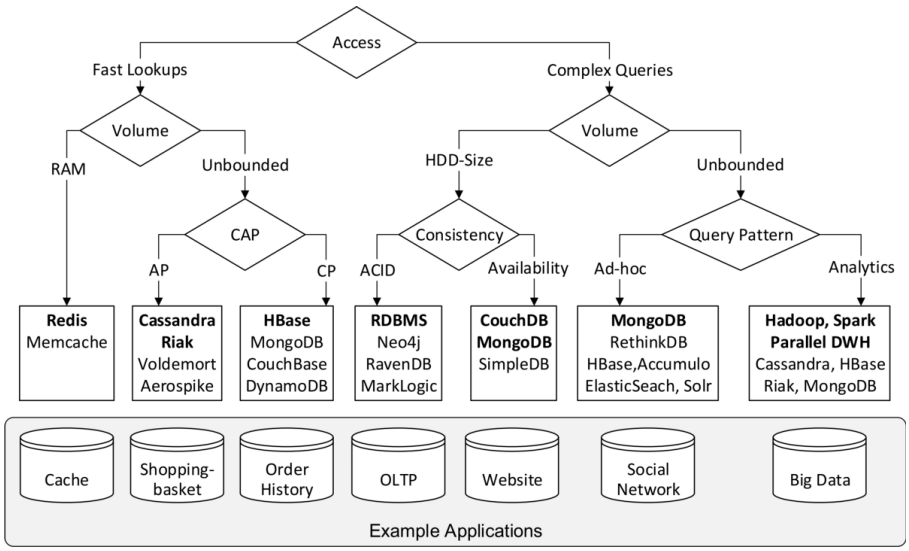
Graph

Nodes and relationships as first-class data

Social networks, recommendations, fraud rings

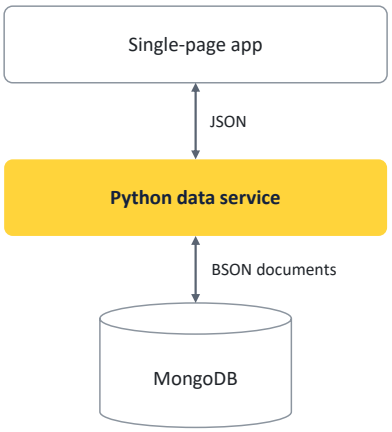
Neo4j

Choosing a Data Store: A Decision Tree



Decision guidance after Gessert et al. (2017), NoSQL database systems: a survey and decision guidance

Document Stores: JSON End to End



a document in the books collection

```
{
  "_id": 42,
  "title": "SICP",
  "authors": ["Abelson", "Sussman"],
  "formats": {"print": 49.9, "ebook": 29.9}
}
```

PyMongo (Python)

```python
from pymongo import MongoClient

with MongoClient("mongodb://localhost:27017") as client:
    book = client.bookstore.books.find_one({"_id": 42})
```

Key idea

The same shape from browser to disk: little mapping code — `find_one` returns a plain `dict`.

QUIZ

Quick Check: Choosing a Data Store

An online shop stores shopping baskets. They are looked up only by session id, traffic is very high, and the store must scale out across many machines. Which family fits best?

- **A** A relational database with joins
- **B** A key–value store such as Redis or DynamoDB
- **C** A graph database such as Neo4j
- **D** A Hadoop data warehouse

ANSWER

Quick Check: Choosing a Data Store

An online shop stores shopping baskets. They are looked up only by session id, traffic is very high, and the store must scale out across many machines. Which family fits best?

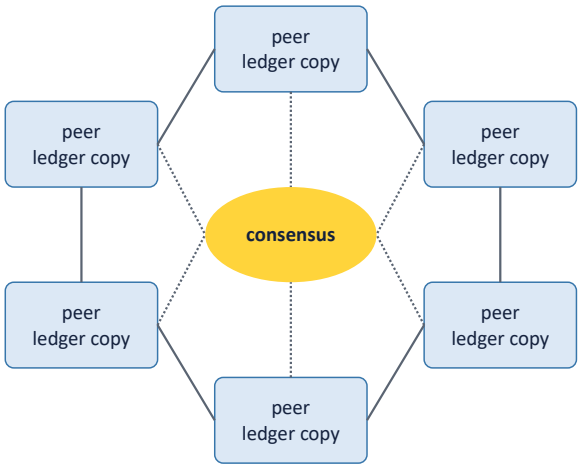
- A — A relational database with joins
- **B — A key–value store such as Redis or DynamoDB**
- C — A graph database such as Neo4j
- D — A Hadoop data warehouse

Why

Access is by a single key, needs fast look-ups and horizontal scale — exactly the "fast look-ups, unbounded volume" branch of the decision tree (the *shopping basket* example).

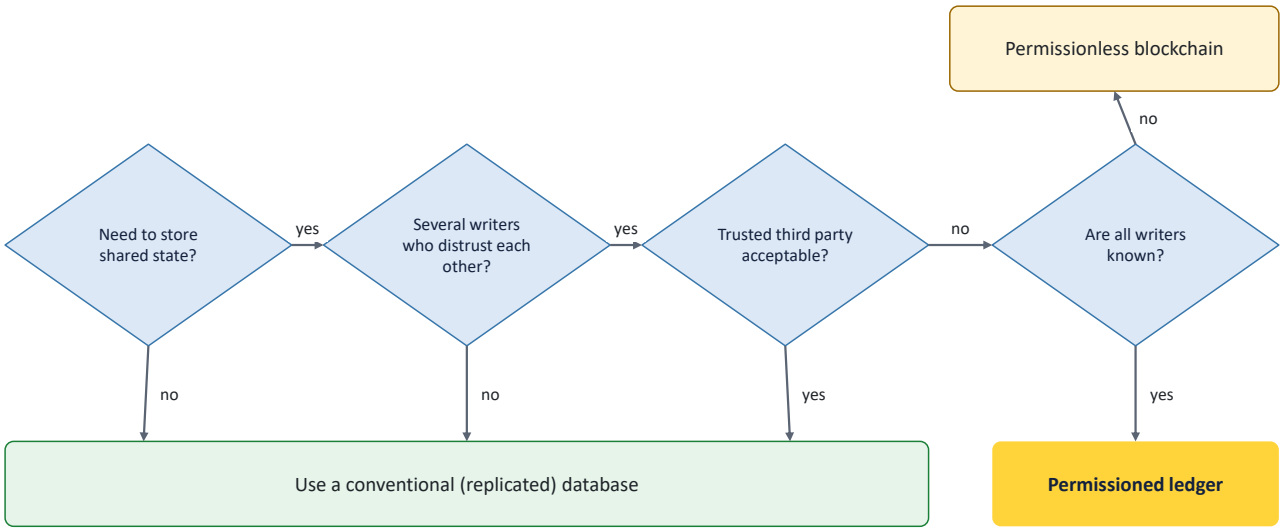
6 · WEB ARCHITECTURES

Distributed Ledger as a Data Service



- A **ledger of transactions**, replicated on many peers
- No single entity controls it; peers agree through **consensus**
- Append-only, hash-chained blocks: tampering is detectable
- Public (anyone can join) or **permissioned** (known members)
- Layers: smart contracts → blockchain (transactions, consensus, P2P) → TCP/IP

Do You Need a Distributed Ledger?



Simplified after Wüst and Gervais (2018), Do you need a blockchain?

07

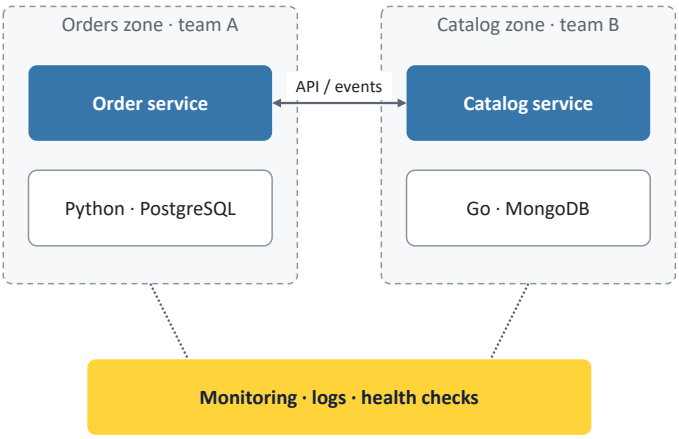
The Evolutionary Architect

Guiding a system that never stops changing

- The architect as a town planner: zoning
- Trade-offs and a principled approach
- Strategic goals, principles and practices
- The Twelve-Factor App and a real-world example

The Architect as Town Planner

Software keeps changing after release, so there is no final blueprint. Like a **town planner**, the architect defines **zones** and the rules between them, and lets the details inside each zone evolve.



Between the zones: the architect's concern

- How services talk to each other
- Monitoring the health of the whole system

Within a zone: the team's choice

- Technology stack and data store
- But ten different stacks make hiring and moving people between teams harder

A Principled Approach to Trade-offs

- System design decisions are **trade-offs**, and microservices give us many of them
- Some decisions can be made on the spot, with all information available
- Others must be made on **incomplete information**: principles guide them

Decision	Option A	Option B	What is traded
Datastore for a new service	Familiar relational DB	Less familiar DB that scales better	Scalability vs. risk and maintainability
Technology stacks	One stack everywhere	Two or three stacks	Fit for purpose vs. hiring and team mobility
Integration style	Synchronous HTTP calls	Asynchronous messaging	Simplicity vs. availability and loose coupling

Key idea

Name what is being traded with the **quality attributes** met earlier in this module (performance, availability, modifiability, …), then let a principle decide.

Strategic Goals, Principles and Practices



Strategic goals

Where the organisation is going

- Set by the business
- e.g. *enter new markets*
- Change rarely

Principles

Rules that align our work with a goal

- e.g. *teams ship independently*
- A handful, not dozens
- Change sometimes

Practices

How we carry out the principles

- Detailed, technology-specific
- e.g. *coding guidelines, central logging*
- Change most often

Tip

Keep principles few (Newman suggests fewer than ten) so everyone can remember them; practices must be concrete enough for any developer to follow.

From Goal to Practice: Two Examples

Strategic goal	Principle	Practices
Decrease time to market for new features	Delivery teams control the full life cycle of their software and ship whenever they are ready, independently of other teams	One CI/CD pipeline per service; each service deployed into an isolated cloud account with self-service resources
Aggressively grow the offering in other countries	The whole system is portable, so it can be deployed locally to respect data sovereignty	Infrastructure as code; no region-specific hard-coding; container images

Note

Practices also reflect organisational constraints: if operations supports only one Linux distribution, the practices must say so. (The original example, CentOS, reached end of life in 2024.)

Principles in Practice: The Twelve-Factor App

Factor	Rule	Factor	Rule
I Codebase	One codebase in version control, many deploys	**VII** Port binding	Export HTTP by binding to a port
II Dependencies	Declare and isolate them (`pyproject.toml`, venv)	**VIII** Concurrency	Scale out by running more processes
III Config	Store config in the environment	**IX** Disposability	Fast start-up, graceful shutdown
IV Backing services	Databases and queues are attached resources	**X** Dev/prod parity	Keep environments as similar as possible
V Build, release, run	Strictly separate the three stages	**XI** Logs	Treat logs as event streams on stdout
VI Processes	Stateless, share-nothing processes	**XII** Admin processes	Run one-off tasks as processes

12factor.net: written at Heroku by Adam Wiggins (2011) for apps on its platform; now a widely used baseline for cloud-native services.

Factor III in Python: Config in the Environment

settings.py

```python
import os

# Twelve-factor: deploy-specific config comes from the
# environment, with safe defaults for a developer laptop.
DATABASE_URL = os.environ.get("DATABASE_URL", "sqlite:///dev.db")
DEBUG = os.environ.get("DEBUG", "0") == "1"

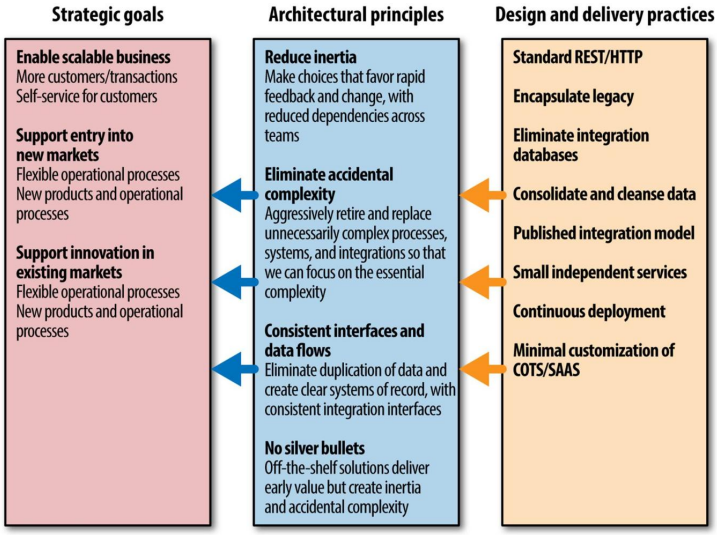
print(f"database: {DATABASE_URL}")
print(f"debug: {DEBUG}")
```

TERMINAL

```
$ python settings.py
database: sqlite:///dev.db
debug: False
$ export DATABASE_URL=postgresql://db/orders
$ DEBUG=1 python settings.py
database: postgresql://db/orders
debug: True
```

- Same build runs in dev, test and production; only the environment differs
- Never commit secrets: inject them at deploy time
- `pydantic-settings` adds types and validation (Module 8)

A Real-World Example



Newman, Building Microservices (2015), Fig. 2-1 — after Evan Bottcher

Key idea

Read right to left: practices support principles, principles serve goals.

- *Small independent services, continuous deployment* → **reduce inertia**
- *Standard REST/HTTP, published integration model* → **consistent interfaces**
- *Encapsulate legacy, eliminate integration databases* → **eliminate accidental complexity**

QUIZ

Quick Check: Goals, Principles, Practices

Which statement is a **practice** rather than a strategic goal or a principle?

A. Enable the business to scale to more customers and transactions

B. Teams own the full life cycle of the services they build

C. Every service writes structured JSON logs to stdout for central collection

D. Make choices that favour rapid feedback and change

ANSWER

Quick Check: Goals, Principles, Practices

Which statement is a **practice** rather than a strategic goal or a principle?

A Enable the business to scale to more customers and transactions

B Teams own the full life cycle of the services they build

C Every service writes structured JSON logs to stdout for central collection

D Make choices that favour rapid feedback and change

Why

Practices are concrete, often technology-specific instructions. A is a strategic goal; B and D are principles that guide decisions without prescribing a technology.

DISCUSS

Zoning Decisions

Your organisation runs 30 Python services. A team wants to build its new **recommendation service** in Rust for performance.

- Which concerns belong to the architect (between zones), which to the team?
- What would make you say yes, and what would make you say no?
- Write one principle and one practice that settle similar requests in the future

Tip

Think in quality attributes: performance vs. maintainability, operability and team mobility.

08

Microservices

Decomposing one application into small, independently deployable services

- Definitions and case studies
- The monolith and its growing pains
- Characteristics, advantages and challenges
- Fallacies of distributed computing, designing for failure
- Decomposition: capabilities, bounded contexts, strangler fig

What Are Microservices?

An **architectural style**, an alternative to the monolith: one application is composed of a suite of **small services**, each of which

Runs in its own process
Not merely a module or component inside one executable

Talks over open protocols
REST over HTTP, gRPC, messaging

Encapsulates a business capability
Rather than language constructs such as packages

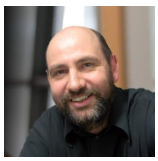
Is independently deployable
Separately written, deployed, scaled and maintained

Is replaceable and upgradable
Can be rewritten without touching its neighbours

May use its own stack
Potentially a different language or data store

! **Watch out**
…with all the benefits **and risks** this implies: a microservice system is a distributed system.

Definitions from the Experts



"…a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API."

James Lewis & Martin Fowler (2014)



"Fine-grained SOA" — more fully: "loosely coupled service-oriented architecture with bounded contexts."

Adrian Cockcroft (Netflix cloud architect)

A current definition

"Microservices are independently releasable services that are modeled around a business domain." — Sam Newman, *Building Microservices*, 2nd ed. (2021)

What Microservices Are Not

Not the same as SOA

SOA integrates many enterprise applications; microservices decompose a single application

Not a silver bullet

They trade in-process simplicity for distributed-system problems: latency, partial failure, consistency

Not new

Processes, services and message passing are decades old: you may already use microservices without the name

Not a goal in itself

Adopt them for a reason: independent deployment, targeted scaling or team autonomy

Common misconception

Micro does not mean tiny: size is the least important property of a service.

Historical Case Studies: eBay, Twitter, Amazon

Company	Started as	Intermediate step	Later architecture
eBay (1995)	Perl monolith written over one Labor Day weekend	C++ monolith: 3.4 million lines in a single DLL	Partitioned Java system, then polyglot microservices (5th generation)
Twitter (2006)	Ruby on Rails monolith	JavaScript + Rails front end, Scala back end	Polyglot services (3rd generation)
Amazon (1994)	C++ monolith	Services in Java and Scala	Polyglot services owned by *two-pizza* teams
Netflix (1997)	Java monolith on one Oracle database	After a 2008 database corruption: move to AWS (2009–2016)	Hundreds of cloud microservices

Note

Historical snapshots, mostly from conference talks of 2010–2016; all four architectures have kept evolving since.

Current Trends: Not a One-Way Street

Many large platforms run microservices

Netflix, Amazon, Uber, Spotify: hundreds to thousands of services, each owned by one team

Big monoliths still exist

Facebook's web tier remains a very large monolithic Hack (PHP-derived) codebase, backed by many services

Some teams moved back

Segment (2018) merged about 140 services into one; Amazon Prime Video (2023) turned a distributed monitoring tool into one process and cut its cost by 90%

The modular monolith

Shopify runs a large Rails monolith with strictly enforced internal module boundaries

Key idea

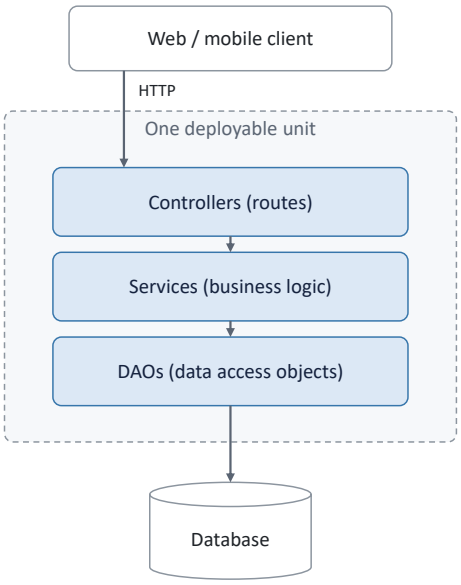
Choose an architecture for your quality-attribute needs and team size, not for fashion.

A Monolithic Shopping Cart

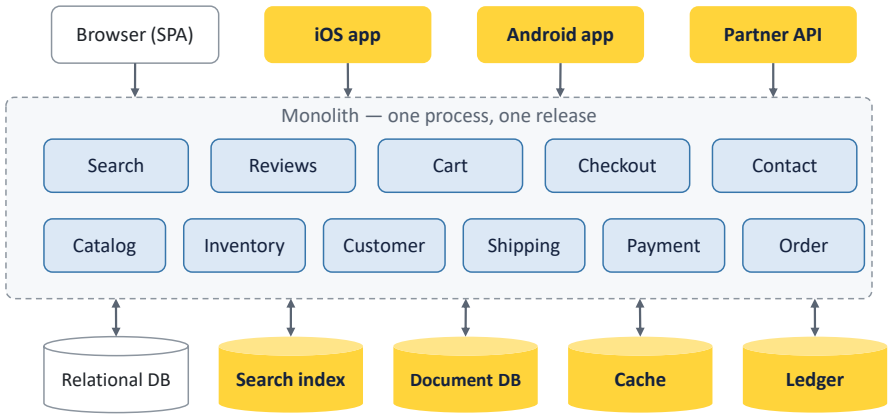
- One application serves the web and mobile interfaces
- Functions: searching, product catalogue, inventory management, shopping cart, checkout, fulfilment
- Classic layers: controllers → services → data-access objects (DAOs)
- Built, tested and deployed as **one unit** on one database

Keep in mind

How would this look with microservices?



The Monolith Grows: More Clients, More Stores

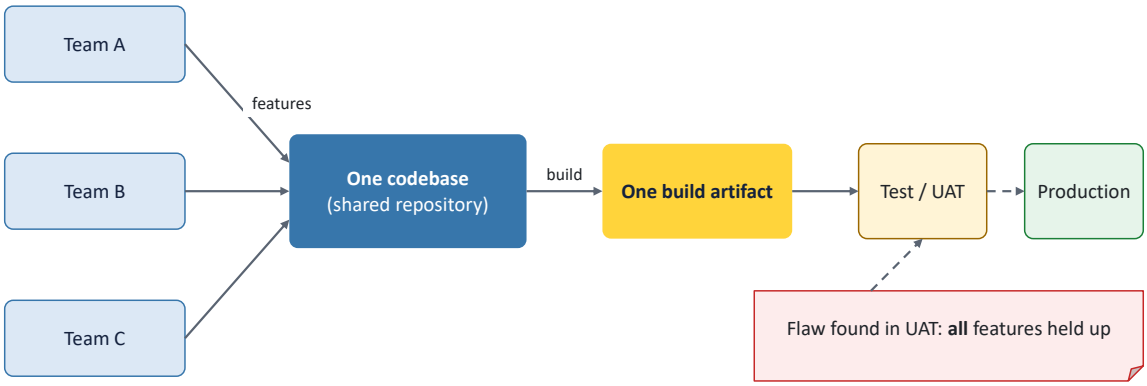


Watch out

Every new client type and every new storage technology (highlighted) must fit into, and be released with, the same deployable.

8 · MICROSERVICES

Monolith Challenges: One Codebase, One Pipeline



Single codebase

Every team merges into one repository; coordination grows with team size

Single deployment

A change to one feature redeploys everything; releases get batched

Single version

Features are *held hostage*: one bug delays unrelated work

8 · MICROSERVICES

Modularity Inside a Python Monolith

project layout

```
shop/
├── app.py          # one entry point
├── db.py           # one shared database
├── catalog/
│   └── service.py
├── cart/
│   └── service.py
├── checkout/
│   └── service.py
└── inventory/
    └── service.py
```

checkout/service.py

```python
# shop/checkout/service.py
from shop.db import session
from shop.inventory.service import reserve


def place_order(cart_id: int) -> None:
    with session() as db:  # one transaction
        items = db.load_cart(cart_id)
        reserve(db, items)  # function call
        db.save_order(cart_id, items)
```

- Single executable, **single language**
- Modularity from language constructs: packages, modules, classes, frameworks
- Calls are fast, in-process and share one transaction

Watch out

Boundaries are conventions only: nothing stops `cart` importing `inventory` internals (tools such as import-linter can enforce rules).

8 · MICROSERVICES

Monoliths: Strengths and Drawbacks

Strengths

- Easy to comprehend (but not to *digest*)
- Easy to test as one unit, up to a size limit
- Easy to deploy and manage as one unit
- Easy to scale by cloning, when care is taken
- Complexity managed by language constructs

Drawbacks

- **Language or framework lock**: one stack, hard to adopt new technology
- **Digestion**: no developer can digest a large codebase; one team cannot manage it
- **Deployment as one unit**: changes are held hostage by other changes
- **All-or-nothing scaling**: a hot module cannot scale alone

Tip

Start with a well-modularised monolith; extract services once a boundary proves stable (Fowler, *MonolithFirst*, 2015).

QUIZ

Quick Check: Monoliths

Which of these is **not** an inherent drawback of a monolith?

A. Every change requires redeploying the whole application

B. The whole application must be scaled together

C. Its code cannot be organised into well-separated modules

D. The whole application is tied to one technology stack

ANSWER

Quick Check: Monoliths

Which of these is **not** an inherent drawback of a monolith?

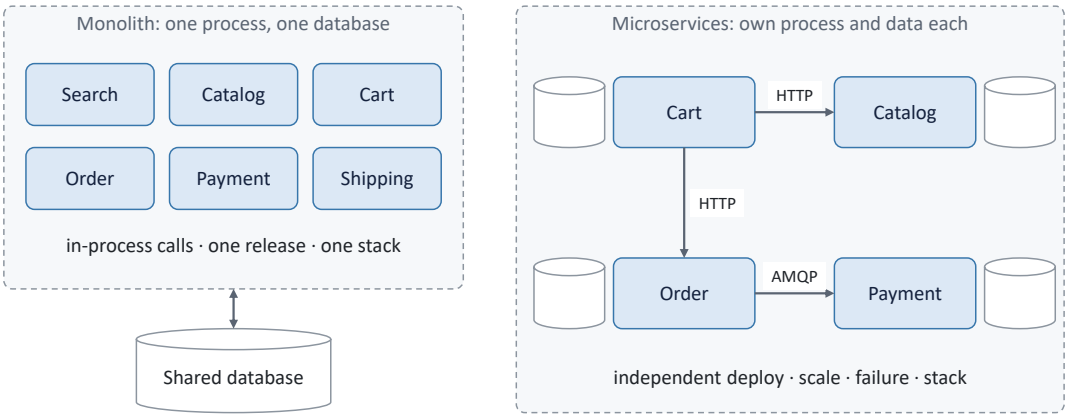
- A — Every change requires redeploying the whole application
- B — The whole application must be scaled together
- C — Its code cannot be organised into well-separated modules
- D — The whole application is tied to one technology stack

Why

A monolith can be highly modular inside (packages, enforced import rules): a *modular monolith*. What it cannot split is deployment, scaling and the technology stack.

8 · MICROSERVICES

Monolith vs Microservices



Key idea

Same business capabilities, different unit of **deployment, scaling and failure**: each service owns its process and its data.

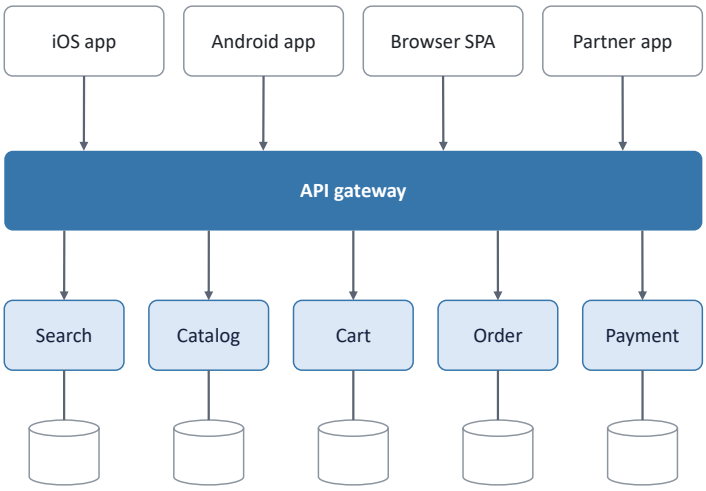
Componentization via Services

	Library component (monolith)	Service component (microservices)
Unit	Package, module, class	Small, independently deployable application
Call	In-process function call (nanoseconds)	Remote call over the network (milliseconds)
Interface	Implicit, easy to bypass with an import	Explicit published contract (e.g. OpenAPI)
Codebase	One codebase, one language	Separate codebases, possibly polyglot
Change	Whole application redeployed	Change scoped to the affected service
Failure	Exception in the same process	Timeout or partial failure

Key idea

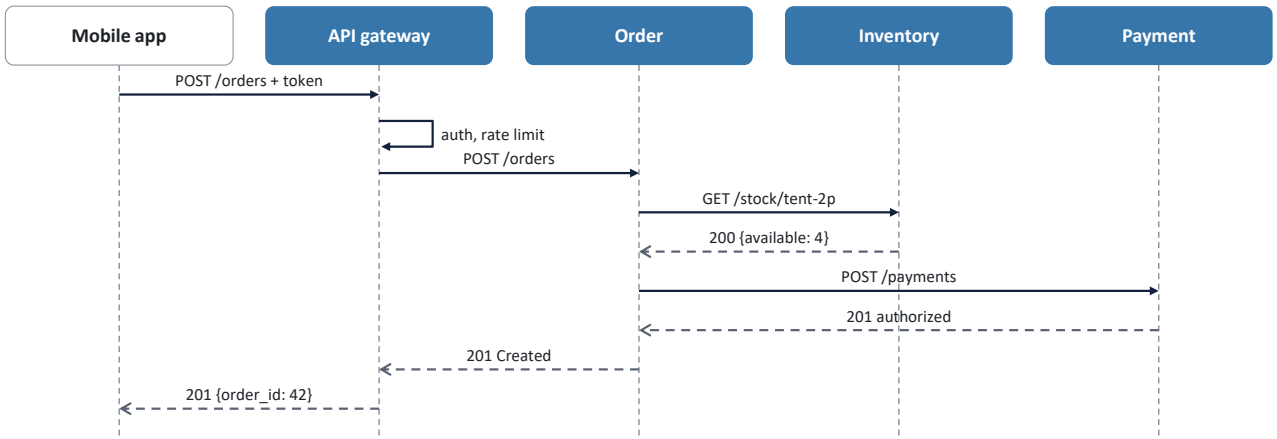
A service is a component that runs *out of process* and is reached by a remote call; this forces the design of clear interfaces (Lewis & Fowler, 2014).

Clients and the API Gateway



- A single entry point for all client types
- Routes `/orders/...` to the Order service, `/search` to Search, …
- Cross-cutting duties: authentication, TLS, rate limiting
- Can aggregate several calls into one response
- Variant: **backend for frontend** (BFF), one gateway per client type

A Request Through the API Gateway



Watch out

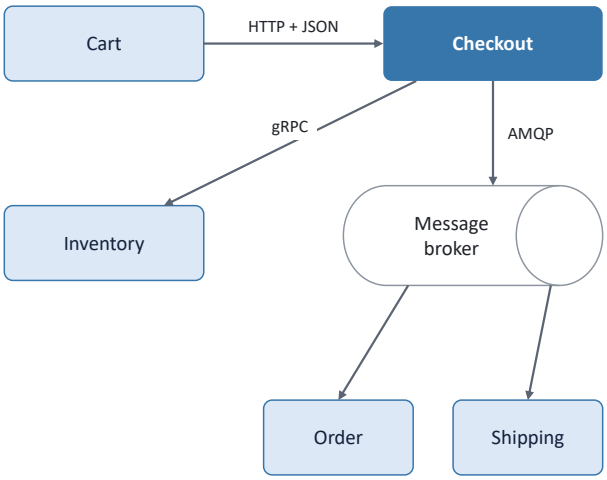
Each arrow between services is a network call that may be slow or fail; the gateway hides this fan-out from the client.

Smart Endpoints, Dumb Pipes

- Lightweight protocols: HTTP, TCP, UDP, WebSocket, messaging (AMQP)
- Payloads: JSON, BSON, XML, Protocol Buffers
- Synchronous request–response (REST, gRPC) or asynchronous messages
- Logic lives in the services; the transport only moves bytes

Key idea

Services integrate through APIs, **never** through a common database (a rule of Netflix's cloud-native architecture).



Organized Around Business Capabilities

- Vertical slices by **business function**: cart, catalogue, checkout
- Not by technology layer; technical services (e.g. e-mail) are practical too
- Each slice suits one **cross-functional team**

> **Note**
> Resource names follow REST conventions (plural nouns): see the REST part of this module and Module 8.

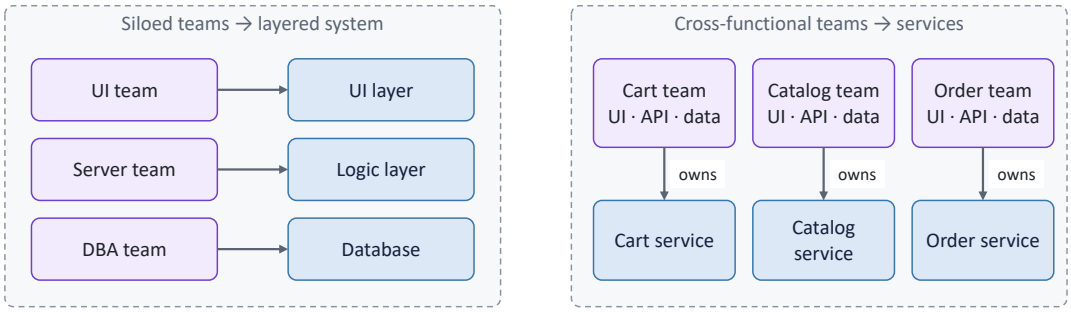
Service	Endpoints
Search	`GET /search?q=tent`
Reviews	`GET /reviews/123` `POST /reviews`

Cart	`POST /carts`
	`GET /carts/123`
	`POST /carts/123/items`
	`PUT /carts/123/items/1`
	`DELETE /carts/123/items/1`
	`DELETE /carts/123`

Contact	`GET /messages/123` `POST /messages`

Conway's Law and Team Ownership

> **Conway's law**
> “Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations.” — Melvin Conway (1968)



- Small services are easy to comprehend, alter, test, version, deploy, replace
- **You build it, you run it**: the team operates its service in production

Decentralized Governance

Right tool for the job

Teams pick language and framework; services evolve at different speeds

Tolerant reader

Read only what you need; ignore unknown fields

Consumer-driven contracts

Consumers publish expectations as tests the provider runs (e.g. Pact)

Choreography, not orchestration

The antithesis of an ESB: no central conductor

tolerant_reader.py

```python
import json

# Order service v2 added a "coupon" field
payload = json.dumps(
    {"id": 7, "total": 59.9, "coupon": "FALL26"})

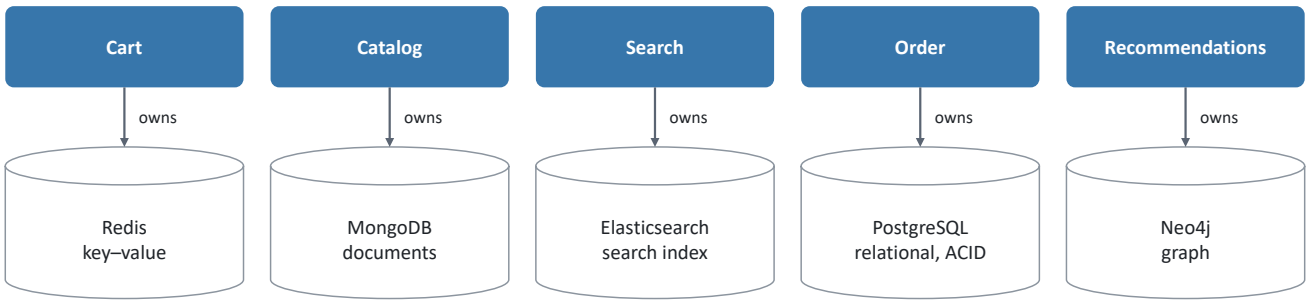

def read_order(raw: str) -> tuple[int, float]:
    """Tolerant reader: take only what we need."""
    data = json.loads(raw)
    return data["id"], data["total"]


print(read_order(payload))
```

OUTPUT

```
(7, 59.9)
```


Decentralized Data: Polyglot Persistence



Freedom

- Best storage technology for each job; don't assume one RDBMS is always best
- Each service **owns** its data; others use its API

Controversy

- No enterprise-wide data model (many DBAs dislike this)
- No distributed ACID transactions: **eventual consistency**, sagas

Microservice Advantages and Quality Attributes

Advantage	Quality attribute	Condition or example
Each service is easy to digest (the whole is harder)	Maintainability	Small codebase, clear boundary
Very easy to test, deploy, version and manage one service	Testability, deployability	One pipeline per service
Scale only the busy service	Scalability	20 search instances, 2 contact instances
Change cycles are decoupled	Modifiability, time to market	No release train
Easier to scale staff	Organisational scalability	Teams work in parallel
No language or framework lock	Evolvability	Rewrite one service at a time
Failures can be isolated	Availability	Only if callers degrade gracefully

Challenges with Microservices

Operational complexity

Complexity moves out of the application into operations: deployment, monitoring, tracing

Partial failure

Services may be unavailable, never a worry in a monolith. Design for failure: *everything fails all the time*

Expensive remote calls

Milliseconds instead of nanoseconds; chatty APIs hurt latency

Data consistency

Transactions rely on **eventual consistency** rather than ACID

Cross-service change

Features span services: interaction, versioning and dependency management get harder

Refactoring boundaries

Moving code across a boundary changes APIs and data ownership

Key idea

Complexity is not removed, only moved: budget for a platform (CI/CD, logging, metrics, tracing) *before* splitting the monolith.

8 · MICROSERVICES

Fallacies of Distributed Computing

1. The network is reliable
Use timeouts, retries and idempotent operations

2. Latency is zero
Avoid chatty calls; batch and cache

3. Bandwidth is infinite
Keep payloads small; paginate and compress

4. The network is secure
Encrypt (TLS, mTLS) and authenticate every call

5. Topology doesn't change
Instances come and go: use service discovery

6. There is one administrator
Many teams and providers: automate, observe

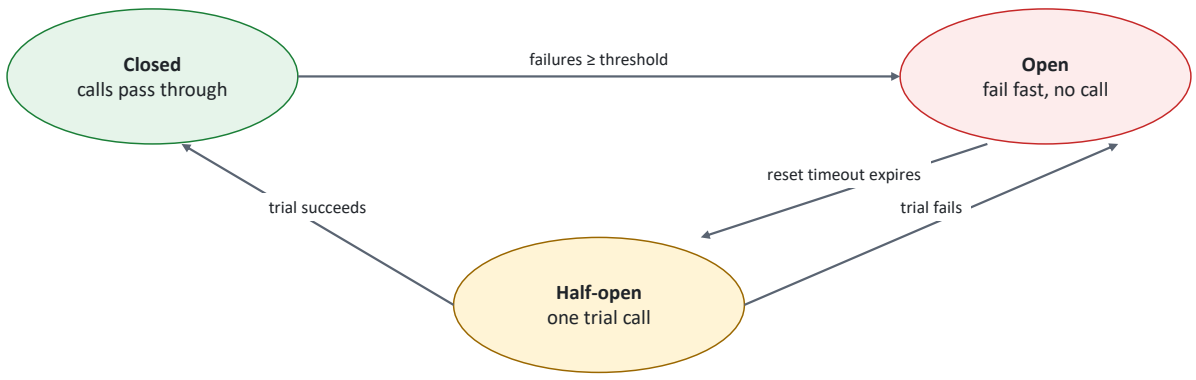
7. Transport cost is zero
Serialisation, bandwidth and infrastructure cost money

8. The network is homogeneous
Stick to standard protocols and formats

Collected at Sun Microsystems by L. Peter Deutsch and colleagues (1994); James Gosling added the eighth (1997).

8 · MICROSERVICES

Designing for Failure: Circuit Breakers



Timeouts
Never wait forever: every remote call gets a deadline, e.g. `httpx.get(url, timeout=2.0)`

Retries with backoff
Retry transient errors with growing delays, only for idempotent operations

Graceful degradation
Fall back: cached recommendations or *stock unknown* instead of an error page

QUIZ

Quick Check: Fallacies

A product page calls the Reviews service 150 times, once per review, one after another. The page takes 4 seconds to load. Which fallacy did the designers fall for?

- A The network is secure
- B Latency is zero
- C Topology doesn't change
- D There is one administrator

ANSWER

Quick Check: Fallacies

A product page calls the Reviews service 150 times, once per review, one after another. The page takes 4 seconds to load. Which fallacy did the designers fall for?

- A The network is secure
- **B Latency is zero**
- C Topology doesn't change
- D There is one administrator

Why

150 sequential round trips of about 25 ms each add up to almost 4 s. Fetch the reviews in one batched call (`GET /reviews?product=123`) or cache them.

8 · MICROSERVICES

How Do You Break a Monolith into Services?

By business capability

What the business *does*: catalogue, pricing, ordering, shipping (mostly **noun**-based)

By use case

What users *do*: checkout, track a shipment (**verb**-based services for key workflows)

Single responsibility at service scale

Gather what changes for the same reason; separate what changes for different reasons (SRP, Module 1)

By subdomain: bounded context

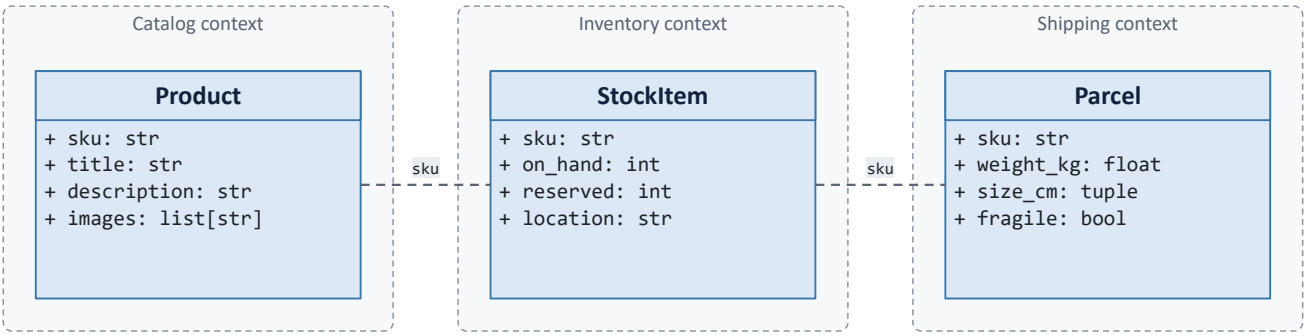
Domain-Driven Design (Evans, 2003): a boundary within which one model and its language are consistent

Key idea

Primary consideration: **business functionality**, not technical layers. A *database service* or *UI service* recreates the monolith's coupling across the network.

8 · MICROSERVICES

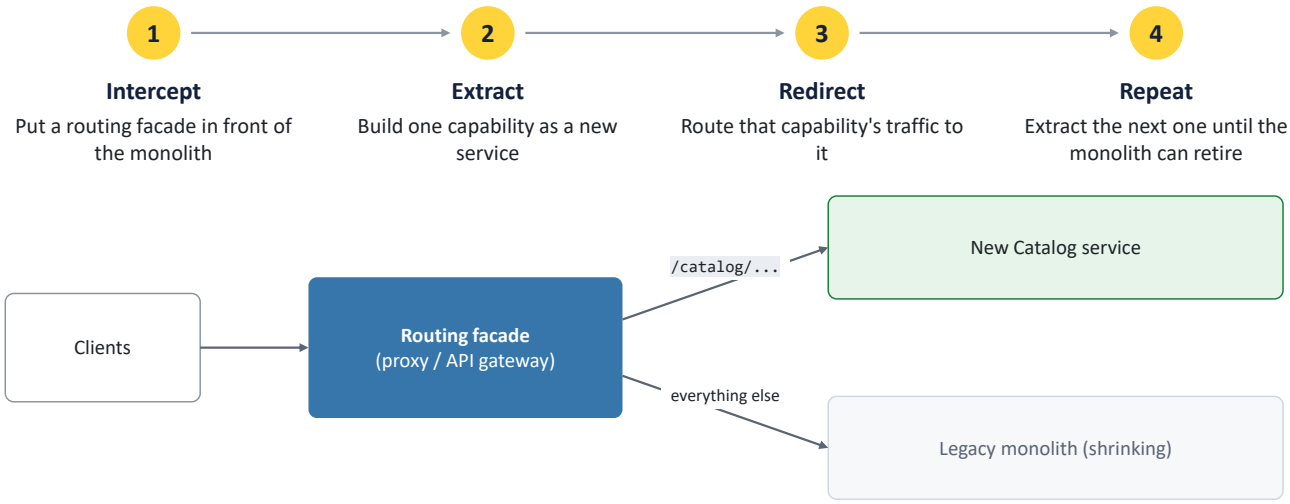
Bounded Contexts: One Word, Several Models



Key idea

Inside a context: one model, one **ubiquitous language**. Across contexts: share only identifiers such as `sku` and translate at the boundary. Each bounded context is a candidate service.

The Strangler Fig Pattern

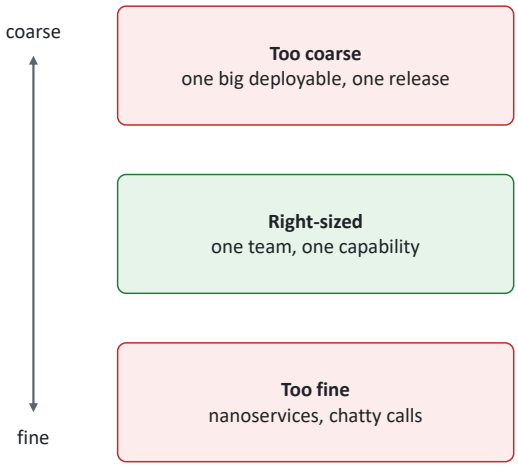


Named after strangler figs, which grow around a host tree until they replace it (Fowler, 2004).

How Micro Is Micro?

- Size is **not** the compelling factor: lines of code are a poor measure
- Small enough for **one developer to digest**
- Small enough to be built and run by **one small team** (Amazon's two-pizza rule)
- Documentation small enough to read and understand
- Predictable and easy to experiment with

> **Watch out**
> Too small leads to *nanoservices*: every feature needs many services, and network overhead dominates.



Two Tiny FastAPI Services: Inventory

inventory.py

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Inventory service")
STOCK = {"tent-2p": 4, "stove": 0}


class Stock(BaseModel):
    sku: str
    available: int


@app.get("/stock/{sku}")
def get_stock(sku: str) -> Stock:
    if sku not in STOCK:
        raise HTTPException(404, "unknown SKU")
    return Stock(sku=sku, available=STOCK[sku])
```

TERMINAL

```
$ uvicorn inventory:app --port 8001
$ curl localhost:8001/stock/tent-2p
{"sku":"tent-2p","available":4}
```

- One program, one process, one port
- Owns its data (here a dict; really its own database)
- Contract published as OpenAPI at `/docs`

Note

FastAPI in depth: Module 8.

Two Tiny FastAPI Services: Orders

order.py

```python
import httpx
from fastapi import FastAPI, HTTPException

app = FastAPI(title="Order service")
INVENTORY_URL = "http://localhost:8001"


@app.post("/orders/{sku}", status_code=201)
def place_order(sku: str) -> dict[str, str]:
    url = f"{INVENTORY_URL}/stock/{sku}"
    try:
        reply = httpx.get(url, timeout=2.0)
        reply.raise_for_status()
    except httpx.HTTPError as exc:
        detail = "inventory unavailable"
        raise HTTPException(503, detail) from exc
    if reply.json()["available"] < 1:
        raise HTTPException(409, "out of stock")
    return {"sku": sku, "status": "accepted"}
```

- A **remote call** with a timeout: never wait forever
- Network failure → `503 Service Unavailable`
- Business rule → `409 Conflict`
- `INVENTORY_URL` belongs in the environment (Factor III)

Note

Replacing this synchronous call with an *OrderPlaced* event on a broker is the topic of Module 9.

DISCUSS

Decompose the Monolithic Shop

The monolith has these modules: **Search, Catalog, Reviews, Cart, Checkout, Order, Inventory, Customer, Payment, Shipping, Contact**.

- Group them into services by business capability
- Name the data each service owns
- Mark which interactions are synchronous calls and which are events
- Which service would you extract first with the strangler fig pattern?

> ✓ **Tip**
> Aim for 6–8 services. Two modules that always change together belong together.

ANSWER

Decompose the Monolithic Shop

Service	Owns	Interactions
Catalog	Products, prices, images	Publishes *ProductChanged*
Search	Search index	Consumes *ProductChanged*
Reviews	Reviews, ratings	Called by product pages
Cart	Carts (Redis)	Calls Catalog for prices
Ordering (checkout + order)	Orders	Calls Inventory and Payment; publishes *OrderPlaced*
Inventory	Stock levels, reservations	Reserve call; consumes *OrderPlaced*
Payment	Payment records	Publishes *PaymentAuthorized*
Customer (+ contact)	Profiles, messages	Called by Ordering and the UI
Shipping	Shipments	Consumes *PaymentAuthorized*

09

Service-Oriented Architecture

Integrating enterprise applications through shared, standards-based services

- The enterprise integration problem
- SOA concepts, characteristics and benefits
- Services: anatomy, SOAP and REST
- Building blocks: registry, ESB, events, BAM
- Standards and the AviTrec case study

The Problem with Enterprise Integration

Companies of every size spread important data across many systems: marketing, CRM, finance, warehouse, business partners…

Misalignment of business and IT

IT systems do not follow how the business works, so every change is slow and expensive

Duplication of data and functionality

The same customer or product data lives in several systems, and they disagree

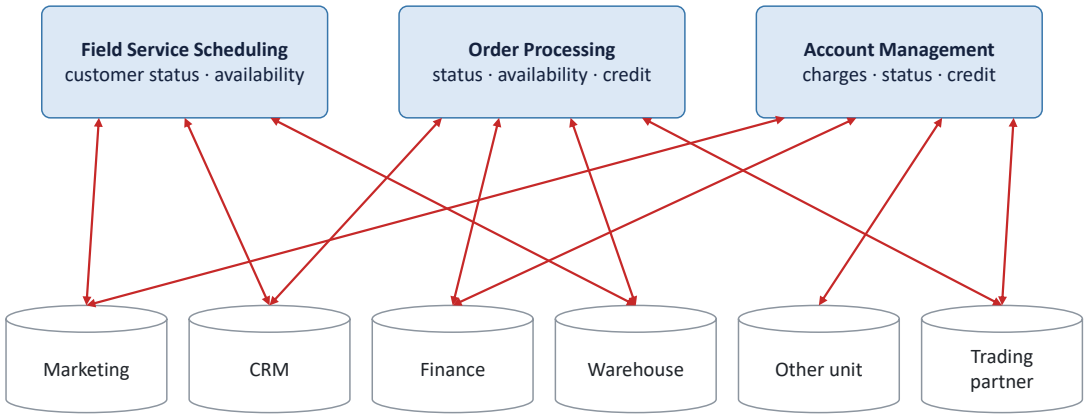
Process and organisational silos

Each department owns its applications; processes that cross departments need glue

★ **Key idea**

Organisations depend on information: IT must stay aligned with the business as markets and regulations change.

Point-to-Point Integration



Watch out

n systems need up to $n(n-1)/2$ connections: 6 → 15, 20 → 190. One interface change ripples through many integrations, and functions such as *customer status* are duplicated.

Defining Service-Oriented Architecture

Definition

SOA is an IT strategy that organizes the discrete functions contained in enterprise applications into interoperable, standards-based services that can be combined and reused quickly, within and between enterprises, to meet business requirements.

Service

A discrete unit of business functionality, available to (ideally) all applications

Described contract

What it does, how to call it, which messages: e.g. a WSDL document

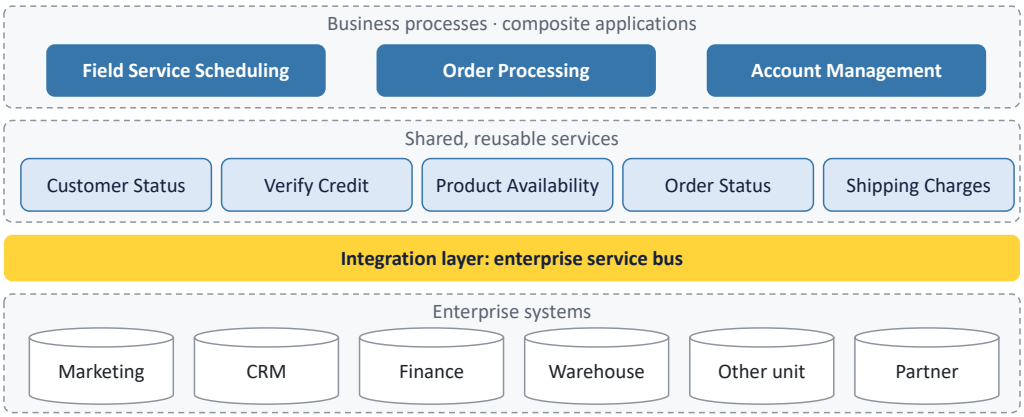
Loose coupling

Consumers are decoupled from providers and their technology

Standard protocols

The *glue* that makes decoupled functions interoperable

SOA Layers: From Systems to Processes



Key idea

Services are assembled into **composite applications**; each service is built once and reused by several processes.

SOA: Characteristics and Benefits

Characteristics

Business-driven

Services are derived from an analysis of business areas and processes

Standards-based

Open standards (W3C, OASIS) make services interoperable

Composition-centric

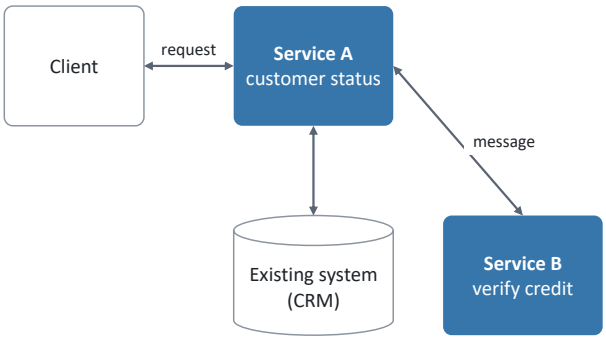
Simple services are aggregated or orchestrated into composite services

Benefits of adopting SOA

- **Business agility**: align IT with the business; remove barriers between units and partners
- **Lower IT cost**: faster delivery; reuse existing infrastructure
- Reuse and consistent interfaces for similar tasks
- Loose coupling: logic in a middle layer
- Scalability: few dependencies between requesters and services

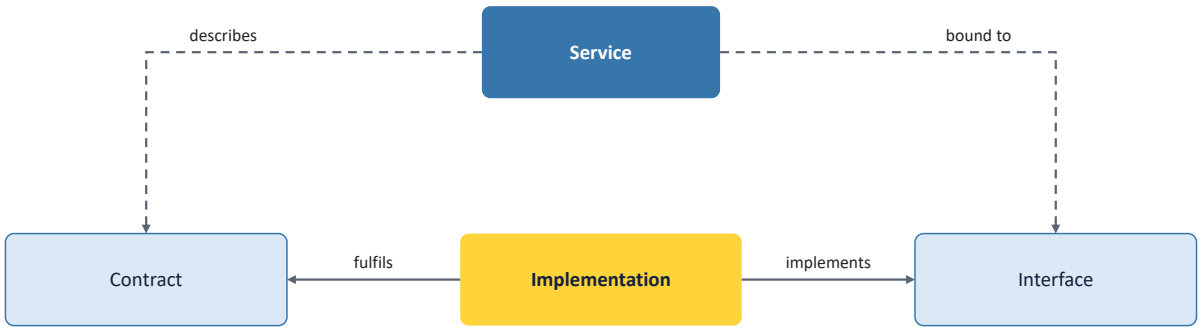
What Is a Service?

- IT's representation of **business functionality**
- Described by a well-defined **interface**
- **Abstracted** from its implementation
- Accessed with standard protocols, the glue for interoperability
- The building block of SOA: self-contained, loosely coupled



interactions are message exchanges

Anatomy of a Service



Element	What it is	Power-grid analogy	Web-service example
Contract	Purpose, functionality, constraints, usage; QoS and SLA	Agreement to pay a rate on a schedule	WSDL or OpenAPI document + SLA
Interface	How consumers invoke the service	Standard outlet and voltage	SOAP endpoint or HTTP routes
Implementation	Deployed code and infrastructure configuration	Power plants and the distribution grid	The Python code behind the routes

SOAP-Based Web Services

- A simple, interoperable **messaging framework** built on XML
- **SOAP**: standard format for messages
- **WSDL**: describes interface, messages and endpoint
- **UDDI**: directory to publish and find services (public registries closed in 2006)
- **WS-***: security, reliability, transactions, management

SOAP 1.2 request

```
<soap:Envelope
    xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
    xmlns:ord="http://avitrec.example/orders">
  <soap:Header/>
  <soap:Body>
    <ord:GetOrderStatus>
      <ord:orderId>42</ord:orderId>
    </ord:GetOrderStatus>
  </soap:Body>
</soap:Envelope>
```


SOAP, REST and gRPC as Service Technologies

	SOAP web services	REST	gRPC
Model	Operations on a service (RPC style)	**Resources** identified by URIs	Remote procedure calls
Contract	WSDL + XML Schema (required)	OpenAPI (optional)	`.proto` file (required)
Payload	XML in a SOAP envelope	Any media type, usually JSON	Protocol Buffers (binary)
Transport	HTTP, JMS, SMTP…	HTTP methods: GET, PUT, POST, DELETE	HTTP/2
Typical today	Legacy enterprise and B2B integration	Public and web APIs	Internal service-to-service calls

Key idea

SOA is an architectural approach; SOAP, REST and gRPC are implementation technologies. REST tries to emulate the architecture of the World Wide Web.

Other SOA Building Blocks

Service registry and repository
Catalogue of services and their metadata: find and reuse

Enterprise service bus (ESB)
Communication backbone: routing, transformation, mediation

Event processing
Publishes events, delivers them to consumers, stores them in queues

Business activity monitoring
Process execution times and KPIs for management

Security
Authentication, authorisation and policies across services

Design and development tooling
Build composite applications, services and processes

Note
Web servers, mail servers and databases are needed too, but these blocks are *unique* to SOA platforms.

Service Registry and Repository

Problem: manage the **life cycle and portfolio** of hundreds of services. The registry and repository store, manage and publish service metadata and govern each service's life cycle.

Provider view
- Track releases of the service
- Track contracts with all consumers
- Apply corporate design policies

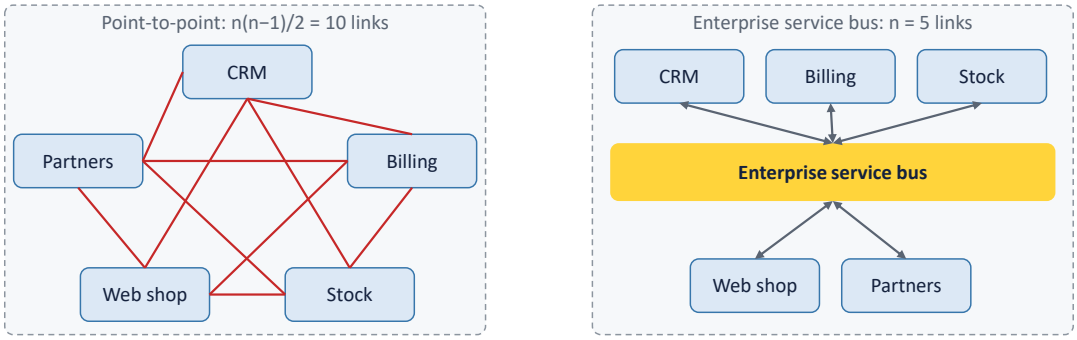
Consumer view
- Find suitable services
- Follow the roadmap of services in use

Enterprise view
- All consumers and services available
- Services identified but not yet built

Note
Modern descendants: **service discovery** at run time (Kubernetes DNS, Consul) and **developer portals / API catalogues** at design time (e.g. Backstage).

Enterprise Service Bus

Middleware that lets disparate partners interact without tight coupling



Routing
Deliver each message to the right service

Transformation
Convert data formats between sender and receiver

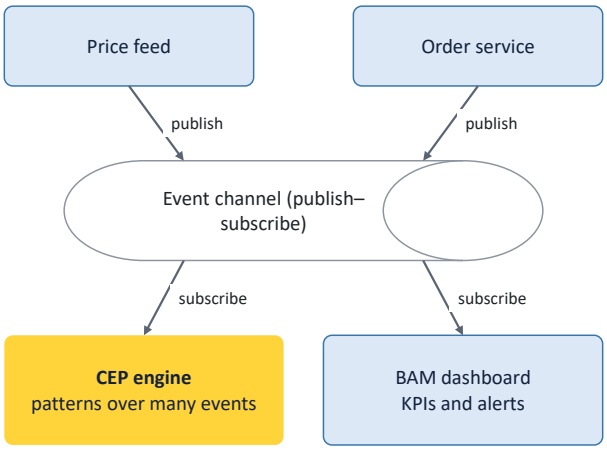
Mediation
Bridge incompatible protocols and middleware

Virtualization
Hide service location and version from consumers

Event-Driven Architecture in SOA

- Problem: detect anomalies, risks or opportunities from business events
- EDA extends SOA with **publish–subscribe** and **asynchronous** events
- **Complex event processing (CEP)**: find patterns across many events, e.g. a price trend rather than one tick

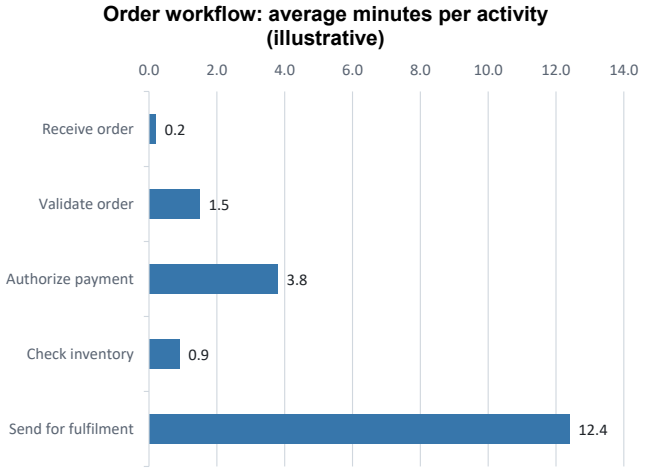
> **Note**
> Full treatment of events, brokers and CQRS: Module 9.



Business Activity Monitoring

- Problems: detect bottlenecks and critical situations early; gain operational insight
- A **real-time operational view** of business processes and activities
- Alerts and notifications when a KPI crosses a threshold

> **Note**
> Today: observability dashboards (e.g. Grafana) over metrics and events.

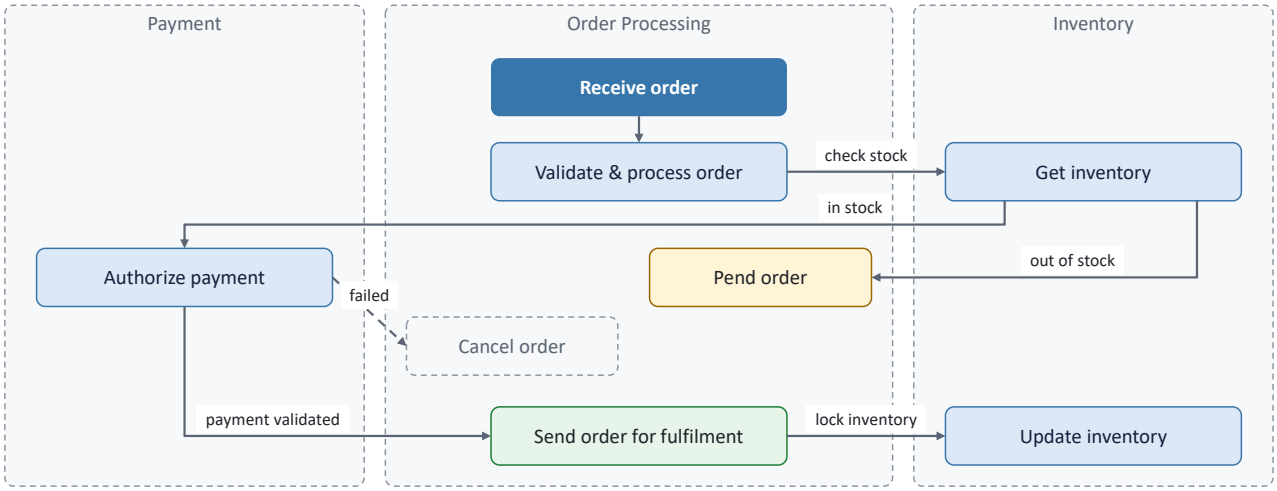


Standards Used in SOA

Category	Classic SOA standards	Modern counterparts
Transport	HTTP(S), IIOP, JMS, SMTP	HTTP/2, AMQP, Kafka protocol
Message	SOAP; XML, JSON	JSON, Protocol Buffers, CloudEvents
Description	WSDL, WADL	OpenAPI, AsyncAPI, `.proto` files
Discovery	UDDI	Service discovery (DNS, Consul), API catalogues
Quality of service	WS-ReliableMessaging, WS-Security	TLS/mTLS, OAuth 2.0, OpenID Connect
Data access	Service Data Objects (SDO)	A database per service behind its API
Business processes	Orchestration: WS-BPEL (BPEL4WS)	BPMN and workflow engines
Assembly model	Service Component Architecture (SCA)	Container images, Kubernetes manifests
Management	WS-Policy, WS-Security	API gateways, service-mesh policies

Case Study: AviTrec Order Management

AviTrec.com (fictional) sells outdoor gear online. To raise customer satisfaction and market share it modernises its IT with SOA. Its order workflow spans three departments:



QUIZ

Quick Check: ESB Capabilities

An order leaves the web shop as XML with a field `custNo`; the CRM expects JSON with `customer_id`. Which ESB capability handles this?

- **A** Routing
- **B** Transformation
- **C** Virtualization
- **D** Service registry lookup

ANSWER

Quick Check: ESB Capabilities

An order leaves the web shop as XML with a field `custNo`; the CRM expects JSON with `customer_id`. Which ESB capability handles this?

- A Routing
- B Transformation
- C Virtualization
- D Service registry lookup

Why

Transformation converts data formats and schemas between sender and receiver. Routing decides *where* a message goes, virtualization hides location and version, and the registry only describes services.

DISCUSS

Design AviTrec's Services

- Name the services you would expose for the order workflow
- Which of them would other processes reuse?
- Which steps would a process engine or the ESB orchestrate?
- Where do events such as *OrderPlaced* or *StockLow* fit?

Tip

Start from the swimlanes: each department is a candidate service owner.

ANSWER

Design AviTrec's Services

Service	Operations	Reused by
Order Management	receive, validate, pend, cancel	Web shop, call centre, partners
Payment Authorization	authorize, capture, refund	Web shop, returns process
Inventory	get stock, lock, update	Warehouse, purchasing, web shop
Customer Status	status, verify credit	Account management, marketing
Fulfilment	send order, track shipment	Customer service

- Orchestration: a BPEL/BPMN *order process* calls the services in workflow order
- Events on the bus: *OrderPlaced*, *PaymentAuthorized*, *StockLow* feed BAM and CEP

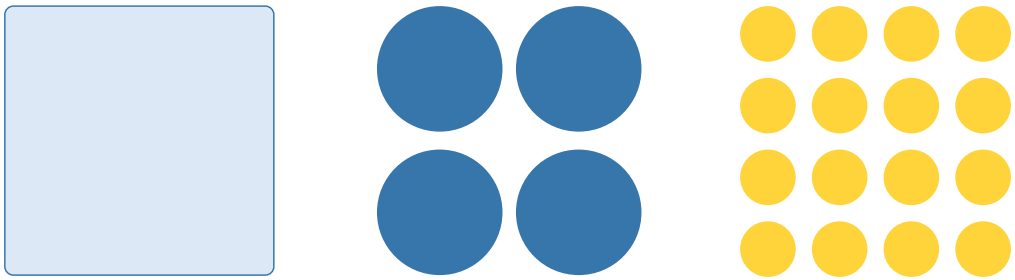
10

Microservices vs SOA

Same service idea, different scope, granularity, sharing and integration

- Granularity: monolith, SOA, microservices
- Scope, integration style, orchestration vs choreography
- Service taxonomies and technology stacks
- Detailed comparison and when to choose which

Monolith, SOA, Microservices: Granularity



Monolithic — single unit

SOA — coarse-grained

Microservices — fine-grained

Monolith

All software components of an application assembled and tightly packaged in one big container

SOA

A collection of services that communicate, passing data or coordinating an activity, to integrate the enterprise

Microservices

One application structured as small autonomous services modelled around a business domain

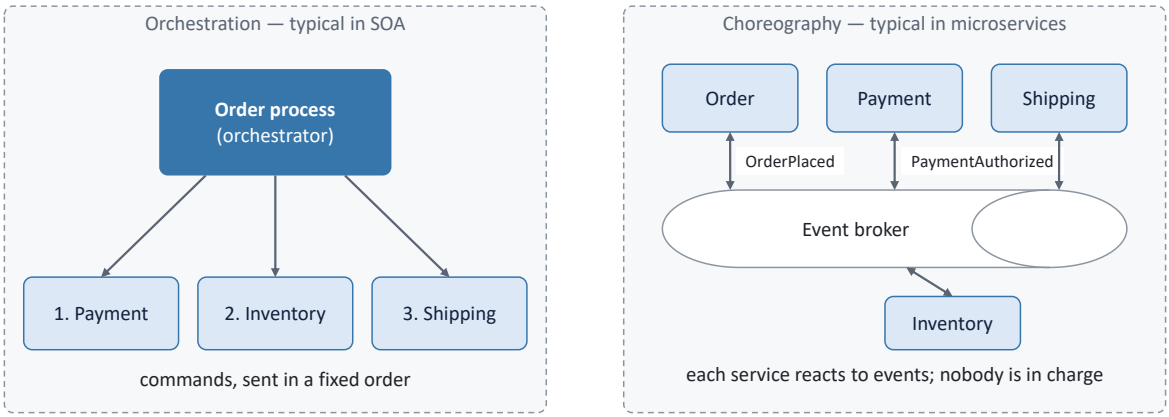
Different Scope, Different Integration Style

	SOA	Microservices
Scope	Integration **between** enterprise systems	Structure **of** one application
Coordination	Orchestration: a central conductor	Choreography: services react to events
Where logic lives	Smart integration technology, dumb services	Smart services, dumb integration technology



Unix pipes: smart filters joined by dumb pipes, the microservices ideal

Orchestration vs Choreography



Key idea

Orchestration makes the process explicit but the conductor becomes a hub of coupling. Choreography keeps services loosely coupled but the flow is implicit: trace it. (Module 9)

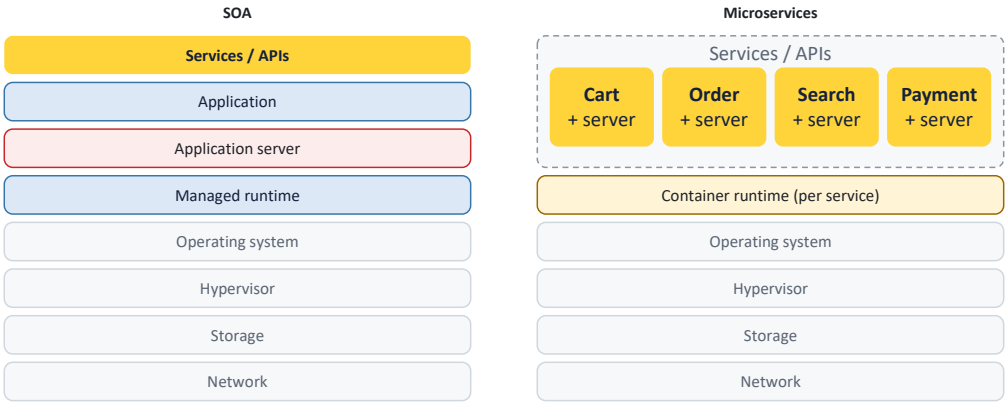
Service Taxonomies Compared

SOA service type	Role	Used by	Microservices
Business (functional)	Coarse-grained core business operations; described in XML, BPEL	Business users	**Functional services**: specific business operations; accessed externally, not shared
Enterprise	Implement business services using application and infrastructure services	Shared-services team	— no shared enterprise layer
Application	Fine-grained, confined to one application; a UI may call them directly	Application teams	— merged into functional services
Infrastructure	Non-functional tasks: authentication, auditing, security, logging	Infrastructure team	**Infrastructure services**: same tasks, internal only

Key idea

SOA: four service types, coordination across all teams. Microservices: two types, minimal coordination.

Technology Stacks Compared



Key idea

Each microservice embeds a lightweight server (e.g. Uvicorn) and ships as a container: no shared application server.

Comparison: Sharing, Granularity, Governance

Dimension	SOA	Microservices
Philosophy	*Share as much as possible*	*Share as little as possible*
Main goal	Reuse of business functionality	Decoupling through **bounded contexts**
Granularity	From small application services to very large enterprise services, even a whole product	Single-purpose services that do one thing really well
Component sharing	A core tenet: enterprise services exist to be shared	Minimised: a component and its data form one unit
Governance	Common governance and standards	People, collaboration, freedom of choice
Request path	Many services plus the ESB: often slower	Fewer hops, though every hop is remote

Comparison: Integration and Communication

Dimension	SOA	Microservices
Middleware	Messaging middleware (ESB): mediation, routing, message enhancement, protocol transformation	An API layer (gateway) between consumers and services, plus simple messaging
Remote access protocols	Many: SOAP, AMQP, MSMQ, JMS…	Few, usually homogeneous: REST over HTTP, gRPC, simple messaging
Heterogeneous interoperability	Promoted: the bus connects mixed protocols	Reduced on purpose: fewer integration choices
Coordination	Orchestration	Choreography

Tip

Many systems with different protocols → consider SOA-style integration. All services reachable through one protocol → microservices are simpler (Richards, 2016).

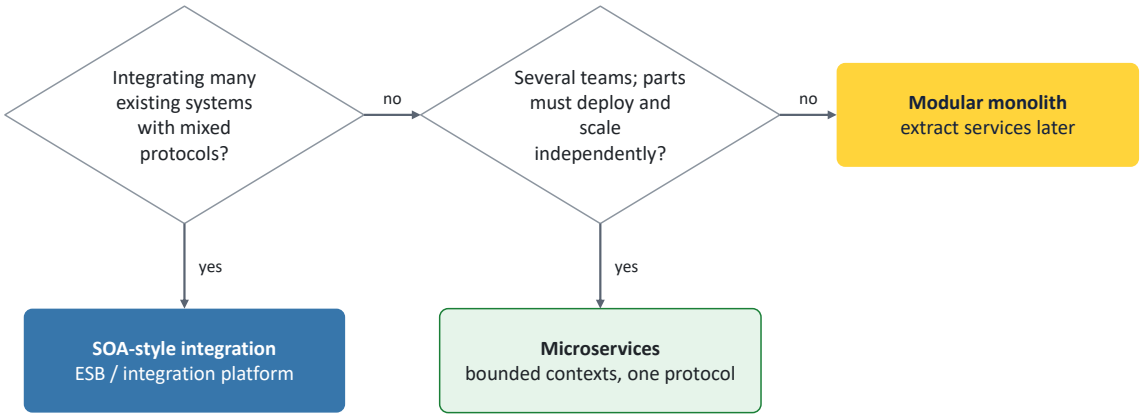
Comparison: Data, Delivery and Runtime

Dimension	SOA	Microservices
Data	Shared enterprise databases, mostly relational	Database per service; polyglot persistence (relational and NoSQL)
Making a change	Often modifies shared enterprise services and every consumer	Create or change one service
Delivery	DevOps and continuous delivery emerging	Strong focus on DevOps, continuous delivery, containers
Runtime	Heavyweight application servers	Lightweight embedded servers; async I/O common

Common misconception

The original table claimed SOA is *multi-threaded* and microservices *single-threaded*. The concurrency model is a choice made per service (threads or event loops, Module 5), not a property of the style.

When to Choose Which



Key idea

Not either–or: an enterprise can integrate its legacy systems SOA-style while building new products as microservices, and start each product as a modular monolith.

QUIZ

Quick Check: Microservices vs SOA

Which statement best characterises microservices **compared with** SOA?

A. Services share as much as possible and communicate through a smart enterprise service bus

B. Services share as little as possible, own their data and use simple, uniform protocols

C. Services are larger and coordinated by a central orchestrator

D. Services must publish SOAP and WSDL contracts

ANSWER

Quick Check: Microservices vs SOA

Which statement best characterises microservices **compared with** SOA?

- A Services share as much as possible and communicate through a smart enterprise service bus
- **B** Services share as little as possible, own their data and use simple, uniform protocols
- C Services are larger and coordinated by a central orchestrator
- D Services must publish SOAP and WSDL contracts

★ **Why**

Microservices minimise sharing through bounded contexts, keep data private to each service and prefer dumb pipes. A and C describe classic SOA; D is not required by either style.

DISCUSS

SOA, Microservices or Modular Monolith?

A · Bank integration

Connect a mainframe, a CRM, a card processor and partner banks that speak SOAP, message queues and file transfers

B · Start-up MVP

Four developers build a new booking app; the domain is still unclear

C · Streaming platform

40 teams ship features daily; video encoding must scale separately from user profiles

✓ **Tip**

Justify each choice with at least two quality attributes.

ANSWER

SOA, Microservices or Modular Monolith?

Scenario	Recommendation	Why
A · Bank	SOA-style integration (ESB or integration platform)	Heterogeneous protocols and existing systems; interoperability and reuse matter most
B · Start-up	Modular monolith first	Small team, unclear boundaries; simplicity and speed; extract services later
C · Streaming	Microservices	Independent deployment and scaling; team autonomy justifies operational cost

10 · MICROSERVICES VS SOA

Practice Problems

Principles and practices
Write one strategic goal, two principles and four practices for a university course-registration system

Twelve-factor audit
Audit a small Flask or FastAPI project against factors III, VI and XI; fix one violation

Decompose a monolith
Split a food-delivery monolith into services by business capability; name each service's data and events

Fallacy hunt
Which fallacies do the two FastAPI services still assume? Add retries with backoff and a fallback

SOA integration
Draw point-to-point and ESB designs for 7 hospital systems; count the links; list the ESB capabilities used

Architecture decision
Write an ADR choosing SOA-style integration, microservices or a modular monolith for a system you know

How to work

For every design answer, name the **quality attributes** you trade and draw the diagram (C4 container level is enough).

MODULE 06 · SUMMARY

Key Takeaways

- ✓ Architecture is the set of significant, costly-to-change decisions that manage a system's complexity
- ✓ Quality attributes drive design; record decisions with views, ADRs and the C4 model
- ✓ Architectural styles (layered, client–server, MVC, pipes and filters, event-driven) and web architectures (REST, data services, NoSQL) are reusable solutions with known trade-offs
- ✓ The evolutionary architect sets zones, principles and practices, and lets the system grow
- ✓ Microservices: independently deployable services around business capabilities, paid for with distributed-system costs
- ✓ SOA integrates enterprise systems through shared services and an ESB; microservices share as little as possible, with smart endpoints and dumb pipes

NEXT UP Module 07 — MongoDB Programming in Python